第3章 基于遗传算法的随机优化搜索

3.1 遗传算法的生物学背景

3.2 遗传算法的产生与发展

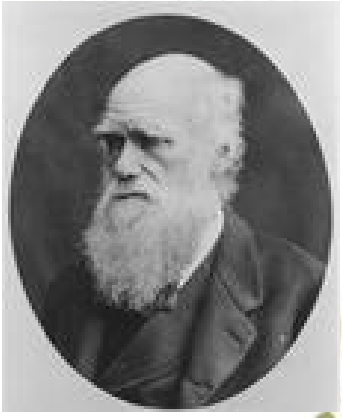
3.3 一些基本概念

3.4 应用举例

3.5 基本遗传算法SGA

3.6 遗传算法的特点及应用

3.7 遗传算法机理的深入探讨



3.1 遗传算法的生物学背景

1. 达尔文的自然选择学说（进化论）

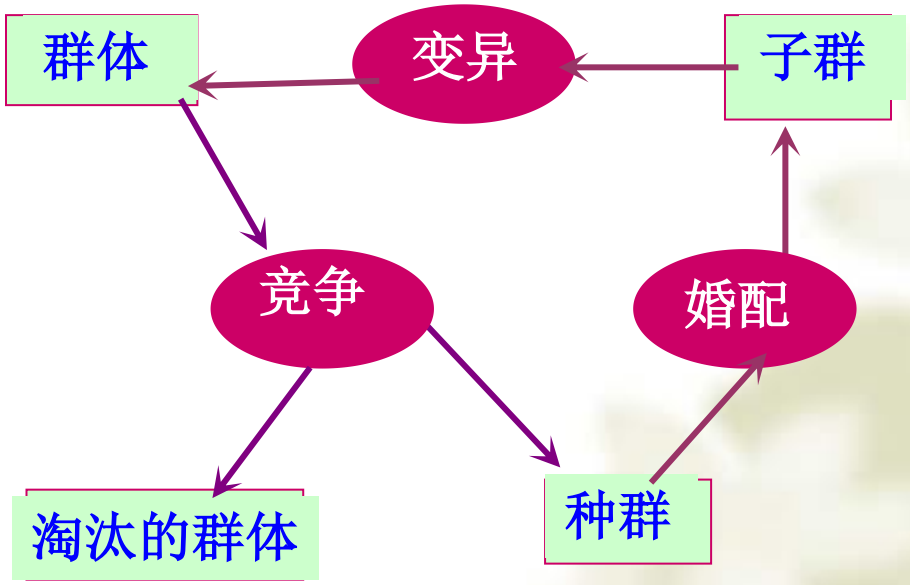
- 在生存斗争中，种群中的个体根据对环境的适应能力而被大自然所选择或淘汰，具有适应性变异的个体被保留，不具适应性变异的个体被淘汰，生物通过个体间的选择、交叉、变异来适应大自然环境，适合自然环境的群体往往有机会产生更多的后代群体，这种过程叫做自然选择。

- 生物群体的生存过程普遍遵循：物竞天择、适者生存，优胜劣汰这一自然选择过程。自然选择过程是长期的、缓慢的、连续的过程。

- 遗传（heredity）：是指父代与子代之间，在性状上存在的相似现象。 ——保证物种的稳定性

- 变异（variation）：是指父代与子代之间，以及子代的个体之间，在性状上或多或少地存在的差异现象。

——是生命多样性的根源



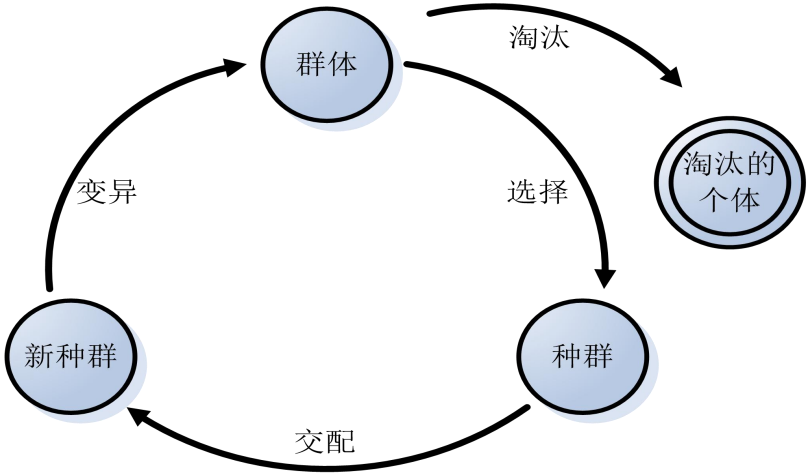
生物进化的基本过程

2. 孟德尔(Mendel) 的遗传学

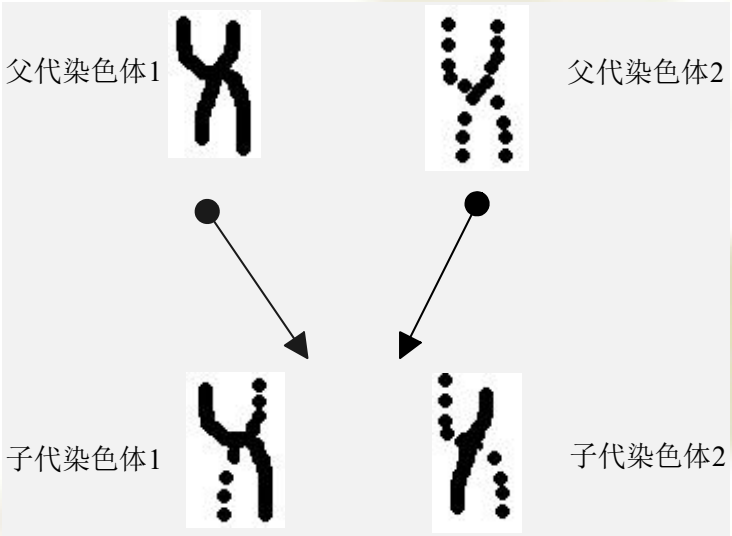


- 孟德尔(1822.7-1884.1)生于奥地利（今捷克），奥古斯丁修道院修道士、神父。遗传学奠基人，代表作 1865《植物杂交试验》。
- 有性繁殖可以使基因不断地进行混合和重组。
- 进化论与遗传学的融合（1930---1947年）

Dobzhansky1937年发表的《遗传学与物种起源》是融合进化论与遗传学的代表作。



生物进化过程



遗传基因重组过程

遗传学基本概念与术语

现代细胞学和遗传学的研究表明，**生物的遗传物质的主要载体是染色体**，染色体主要是由DNA和蛋白质组成，其中**DNA**又是**最主要的遗传物质。**

● **染色体(chromosome)（基因串）：**遗传物质的载体。

染色体是携带基因信息的数据结构，简称个体，**由复数个基因组成，基因是控制生物性状的遗传物质的功能单位和结构单位。**

● **基因(gene)（遗传因子）：基因是有遗传效应的染色体的一个片段，DNA或RNA长链结构中占有一定位置的基本遗传单位，储存着遗传信息。**

● **脱氧核糖核酸(DNA)：**大分子有机聚合物，双螺旋结构。

生物体自身通过对基因的复制和交叉（基因分离、基因自由组合和基因连锁互换）的操作使其性状的遗传得到选择和控制。同时，通过基因重组、基因变异和染色体在结构和数目上的变异产生丰富多采的变异现象。

- **选择**（selection）：指决定以一定的概率从种群中选择若干个体的操作 ；
- **复制**（reproduction）：细胞在分裂时，遗传物质DNA通过复制而转移到新产生的细胞中，新的细胞就继承旧细胞的基因;
- **交叉**（crossover）：在两个染色体的某一相同位置处DNA被切断，其前后两串分别交叉组合形成两个新的染色体。又称基因重组，俗称“杂交”；
- **变异**（mutation）：在细胞进行复制时可能以很小的概率产生某些复制差错，从而使DNA发生某种变异，产生出新的染色体，这些新的染色体表现出新的性状

3.2 遗传算法的产生与发展

“活的有机体是解决问题的专家。它们所表现出来的各种才能足以使最好的计算机程序自惭形秽。这种现象尤其令计算机科学家们感到痛楚。计算机科学家们为了某种算法可能花费数月乃至数年的脑力劳动，而有机体则能通过进化和自然选择这样一种显然并非定向进行的机制获得这种能力。” --- John Holland



遗传算法思想源于自然界“自然选择”和“优胜劣汰”的进化规律，通过模拟生物进化中的自然选择和交叉变异寻找问题的全局最优解。

它最早由美国密歇根大学教授John H. Holland提出，现在已经广泛应用于各种工程领域的优化问题之中。

遗传算法(Genetic Algorithm，GA) 是借鉴生物界自然选择原理和生物遗传机制，通过人工方式构造的一类随机优化搜索算法，是对生物进化过程进行的一种数学仿真，是进化计算的一种最重要形式。

遗传算法VS 生物进化

遗传算法是由美国Michigan大学的J . Holland教授于1975年首先提出的*“Adaption in Natural and Artificial Systems”*。

如何借鉴？

- 自然选择和有性繁殖实际上是生物体的优化过程，正是这种优化过程的不断进行才导致了生物的进化。
- 遗传算法就是一种更为宏观意义下的模拟，它模仿的机制是一切生命和智能的产生与进化过程。

模拟达尔文“优胜劣汰、适者生存”的原理激励好的结构。

模拟孟德尔遗传变异理论在迭代过程中保持已有结构，同时寻找更好的结构。

生物进化与遗传算法对应关系

生物进化	遗传算法
生存能力	适应函数
适者生存	适应函数值最大的解被保留的概率最大
个体	问题的一个解
染色体	解的编码
基因	编码的元素
群体	被选定的一组解
种群	根据适应函数选择的一组解
交叉	以一定的方式由双亲产生后代的过程
变异	编码的某些分量发生变化的过程

- 60年代中期，美国Michigan大学的J. H. Holland教授提出借鉴生物自然遗传的基本原理用于自然和人工系统的自适应行为研究和串编码技术。1967年，他的学生J. D. Bagley在博士论文中首次提出“遗传算法”一词；
- 1975年，Holland出版了著名的“Adaptation in Natural and Artificial Systems”，标志遗传算法的诞生。
- 1985年，在美国召开了第一届遗传算法国际会议，并且成立了国际遗传算法学会(ISGA，International Society of Genetic Algorithms)。
- 1989年，*Holland*的学生*D. J. Goldherg*出版了“Genetic Algorithms in Search, Optimization, and Machine Learning”，对遗传算法及其应用作了全面而系统的论述；
- 1991年，*L. Davis*编辑出版了《遗传算法手册》，其中包括了遗传算法在工程技术和社会生活中大量的应用实例。

1975年，*Holland*出版了第一本系统论述遗传算法和人工自适应系统的专著《自然系统和人工系统的自适应性》。*Holland*也被视为遗传算法的创始人。

第一次系统地论述了遗传算法的基本理论与方法，提出了遗传算法的基本定理——模式定理（schema theorem），从而奠定了遗传算法的理论基础。

模式定理揭示了群体中优良个体（较好模式）的样本数将以指数规律增长，确认了结构重组遗传操作对于获得隐并行性的重要性，从理论上保证了遗传算法是一个可以寻求最优可行解的优化过程。

20世纪80年代，Holland教授实现了第一个基于遗传算法的机器学习系统—分类器系统，开创了基于遗传算法的机器学习的新概念。

3.3 遗传算法的一些基本概念

1. 个体与种群

●个体（individual）

遗传学中是指染色体带有特征的实体；遗传算法中，个体就是模拟生物个体而对问题中的对象（一般就是问题的解）的一种称呼，一个个体就是搜索空间中的一个点。

●种群（population）

模拟生物种群而由若干个体组成的集合，它一般是整个搜索空间的一个很小的子集。该集合内**一定数量的个体组成了种群，群体中个体的数目称为群体规模。**

2. 适应度与适应度函数

●适应度(fitness)就是借鉴生物个体对环境的适应程度，而对问题中的个体对象所设计的表征其优劣的一种测度，适应度的定义与具体问题有关。

● 适应度函数(fitness function)

适应度函数用来评估个体的优劣，作为对个体进行遗传操作的依据。适应度模拟“适者生存”的自然进化律，该函数就是遗传算法中指导搜索的评价函数。适应度函数要有效反映每一个染色体与问题的最优解染色体之间的差距。例如，TSP的目标是路径总长度为最短，自然地，路径总长度就可作为TSP问题的适应度函数。

●**进化**（**evolution**）：生物在其延续生存的过程中，逐渐适应其生存环境，使得其品质不断得到改良，这种生命现象称为进化。遗传算法中进化体现在对解的搜索趋于最优。

3. 染色体及其编码

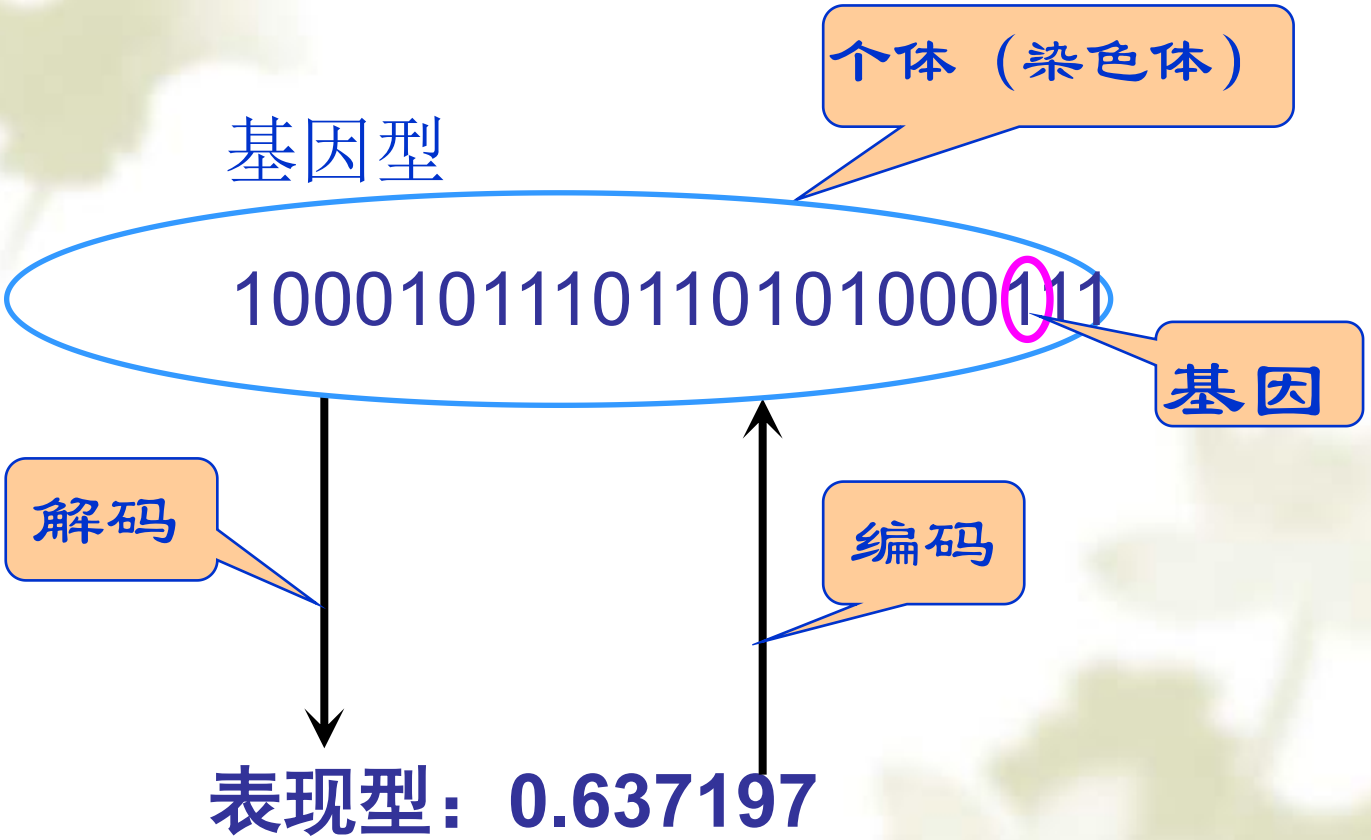
染色体（Chromosome）就是问题中个体对象（即可能解）。而一个**染色体**可以看作是由若干基因组成的位串。所以需要将问题中的个体对象编码为某种位串的形式。这样原个体对象也就相当于生物体的**表现型**，而其编码即染色体也就相当于生物体的**基因型**。在遗传算法中，染色体一般用字符串表示，字符串中的字符就称为**基因**（gene）。

例如：　个体　　染色体

9　----　01001

（2，5，6）----　010 101 110

基因型与表现型



- 生物染色体用数学方式或计算机方式来体现就是一串数码，仍叫染色体，有时也叫个体；
- 遗传算法处理的是染色体，或者叫基因型。

4. 编码与解码

许多应用问题的结构很复杂，但可以化为简单的位串形式编码表示。把位串形式编码表示叫染色体。

将问题结构变换为位串形式编码表示的过程叫编码；而将位串形式编码变换为原问题结构的过程叫解码或译码。

- 编码（coding）：表现型到基因型的映射；
- 解码（decoding）：从基因型到表现型的映射。

主要的编码方法有：二进制编码、**浮点数编码方法、格雷码、符号编码方法、多参数编码方法等。**

5. 遗传操作

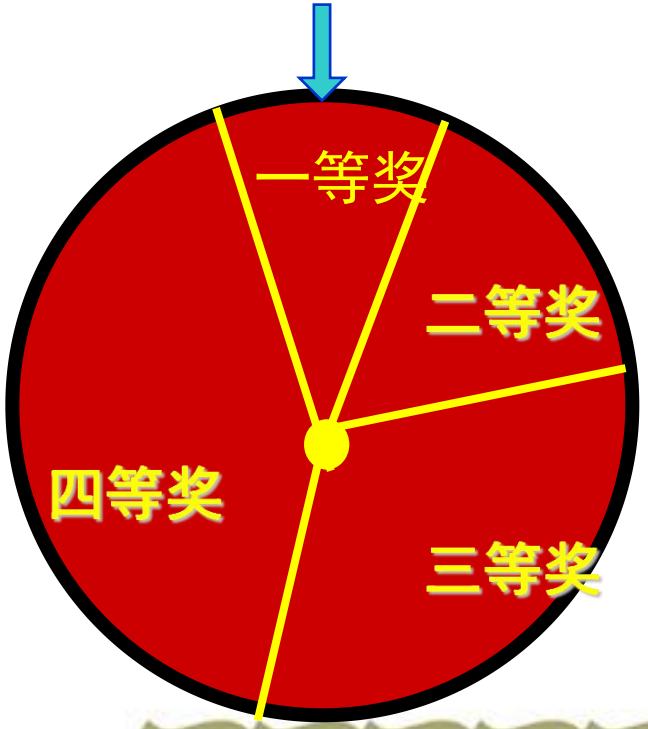
亦称遗传算子(genetic operator)，就是关于染色体的运算。遗传算法中有三种遗传操作:

- 选择-复制(selection-reproduction)
- 交叉(crossover，亦称交换、交配或杂交)
- 变异(mutation，亦称突变)

遗传操作之——选择-复制

通常做法是：对于一个规模为N的种群S，按每个染色体$x_i \in S$的选择概率$P(x_i)$所决定的选中机会，分N次从S中随机选定N个染色体，并进行复制。从当前种群中选出优良个体，使它们有机会作为父代产生后代个体。判断个体优良的依据就是它们的适应度。个体适应度越高，被选择的机会就越多。所有选择出的个体组成配对池。赌轮选择就是基于适应值比例的选择中比较常用的选择策略 。

- 轮盘赌选择又称比例选择算子，其**基本思想**是：各个个体被选中的概率与其适应度函数值成正比。
- 设群体大小为N，　f为适应度函数，$f(x_i)$为个体x_i的适应度，染色体x_i被选中的概率就是其适应度$f(x_i)$所占种群中全体染色体适应度之和的比例。
- 个体x_i的选择概率为用如下公式计算：



$$P(x_i)=\frac{f(x_i)}{\sum_{j=1}^{N} f(x_j)}$$

轮盘赌选择算法

（1）计算群体中所有个体的适应度值；

（2）计算每个个体的选择概率并据此将一个圆盘分成N份；

（3）计算积累概率；

（4）采用模拟赌盘操作（即生成0到1之间的随机数r与每个个体遗传到下一代群体的概率进行匹配）来确定各个个体是否遗传到下一代群体中。即若满足 $\sum_{j=1}^{i-1} p_j < r \le \sum_{j=1}^{i} p_j$ 则选择个体i

轮盘赌选择法可用如下过程模拟来实现：

(1)在［0，1］内产生一个均匀分布的随机数r。

(2)若$r \leqslant q_1$，则染色体x_1被选中。

(3)若$q_{k-1} < r \leqslant q_k$ $(2 \leqslant k \leqslant N)$，则染色体$x_k$被选中。

其中的q_i称为染色体x_i（i=1，2，…，n)的积累概率，其计算公式为

$$q_i = \sum_{j=1}^{i} P(x_j)$$

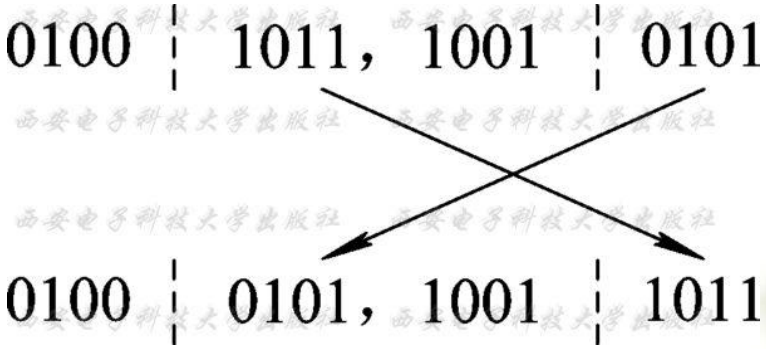
遗传操作之—— 交叉

将群体p(*t*)内各个个体随机搭配成对，对每个个体中交换一部分染色体。一般的遗传算法都有一个杂交率的参数，范围一般是0.6~1，这个杂交率反映两个被选中的个体进行杂交的概率。例如，杂交率为0.8，则80%的“夫妻”会生育后代。

每两个个体通过杂交产生两个新个体，代替原来的“老”个体，而不杂交的个体则保持不变。杂交父母的染色体相互交错，从而产生两个新的染色体，第一个个体前半段是父亲的染色体，后半段是母亲的，第二个个体则正好相反。不过这里的半段不是真正的一半，这个位置叫做杂交点，也是随机产生的，可以是染色体的任意位置。

交叉 就是互换两个染色体某些位上的基因。

例如，设染色体 s_1=01001011，s_2=10010101，交换其后4位基因，即



s_1'=01000101，　s_2'=10011011

可以看做是原染色体s_1和s_2的子代染色体。

遗传操作之—— 变异

变异 就是改变染色体某个(些)位上的基因。**一般遗传算法都有一个固定的变异常数，通常是 0.01或者更小，这代表变异发生的概率。根据这个概率，新个体的染色体随机的变异，通常就是改变染色体的一个字节（0变到1，或者1变到0）。**

例如，设染色体 s=11001101，

将其第三位上的0变为1, 即

$$s=11\underline{0}01101 \rightarrow 11\underline{1}01101= s'。$$

s'也可以看做是原染色体s的子代染色体。

3.4 应用举例

例3.1 利用遗传算法求解区间［0,31］上的二次函数$y=x^2$的最大值。

[分析]

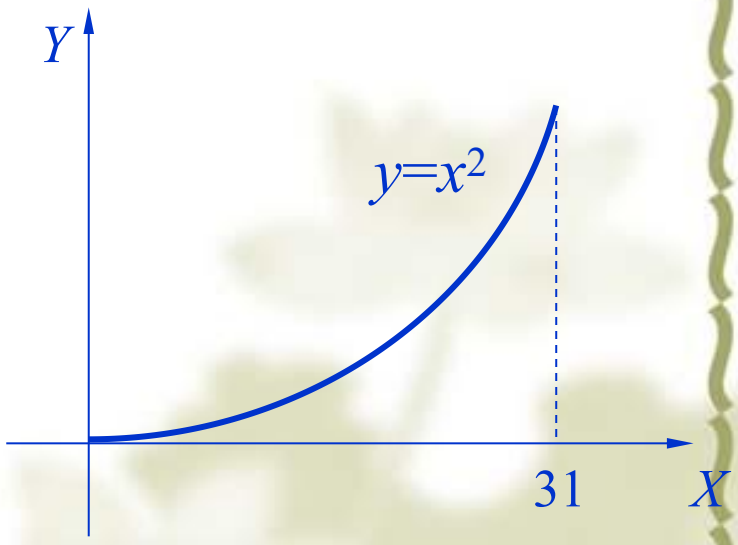
原问题可转化为在区间［0，31］中搜索能使y取最大值的点a的问题。

个体：［0，31］中的任意点x

适应度：函数值$f(x)=x^2$

解空间：区间［0，31］

这样，只要能给出个体x的适当染色体编码，该问题就可以用遗传算法来解决。



解 (1) 设定种群规模,编码染色体，产生初始种群。

将种群规模设定为4；

用5位二进制数编码染色体；

取下列个体组成初始种群S_1

s_1= 13 (01101), s_2= 24 (11000)

s_3= 8 (01000), s_4= 19 (10011)

(2) 定义适应度函数, 取适应度函数 $f(x)=x^2$

(3) 计算各代种群中的各个体的适应度, 并对其染色体进行遗传操作,直到适应度最高的个体，即31（11111）出现为止。

首先计算种群S_1中各个体

s_1= 13(01101), s_2= 24(11000)

s_3= 8(01000), s_4= 19(10011)

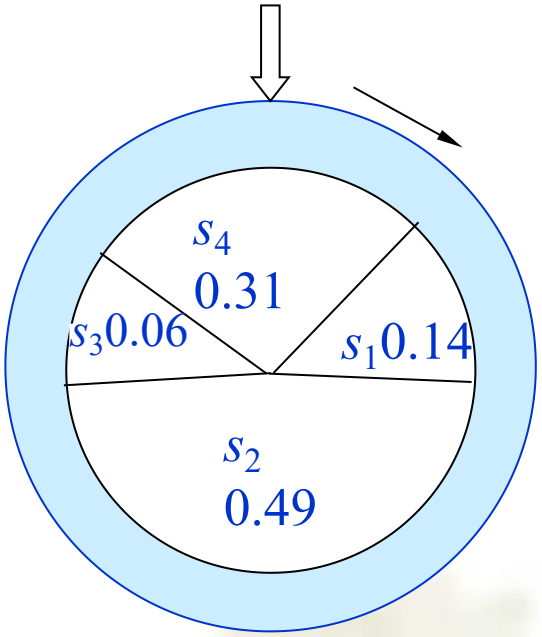
的适应度$f(s_i)$。容易求得

$$f(s_1) = f(13) = 13^2 = 169$$

$$f(s_2) = f(24) = 24^2 = 576$$

$$f(s_3) = f(8) = 8^2 = 64$$

$$f(s_4) = f(19) = 19^2 = 361$$



赌轮选择示意

再计算种群S_1中各个体的选择概率。选择概率的计算公式为

$$P(x_i) = \frac{f(x_i)}{\sum_{j=1}^{N} f(x_j)}$$

由此可求得 $P(s_1) = P(13) = 0.14$

$$P(s_2) = P(24) = 0.49$$

$$P(s_3) = P(8) = 0.06$$

$$P(s_4) = P(19) = 0.31$$

在算法中赌轮选择法可用下面的子过程来模拟:

① 在［0, 1］区间内产生一个均匀分布的随机数r。

② 若$r \le q_1$,则染色体x_1被选中。

③ 若$q_{k-1} < r \le q_k (2 \le k \le N)$, 则染色体$x_k$被选中。 其中的$q_i$称为染色体$x_i$ (i=1，2，…，n)的**积累概率**, 其计算公式为

$$q_i = \sum_{j=1}^{i} P(x_j)$$

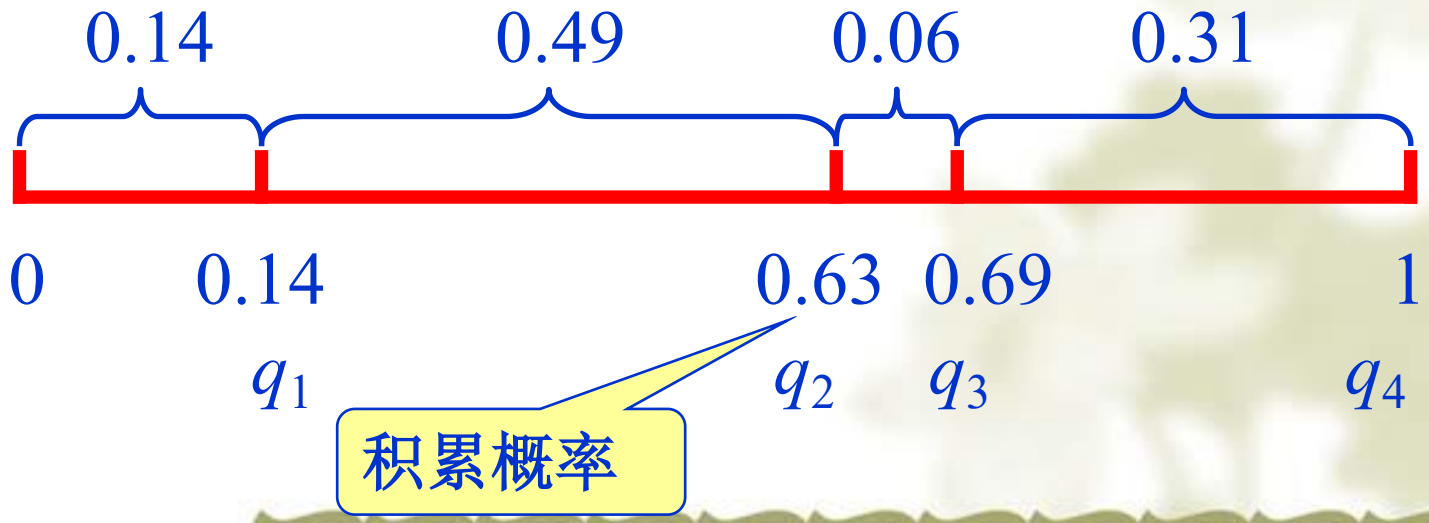
再计算种群S_1中各个体的积累概率 $q_i = \sum_{j=1}^{i} P(x_j)$

$$q(s_1) = \sum_{j=1}^{N} p(s_j) = 0.14$$

$$q(s_2) = \sum_{j=1}^{N} p(s_j) = 0.14 + 0.49 = 0.63$$

$$q(s_3) = \sum_{j=1}^{N} p(s_j) = 0.14 + 0.49 + 0.06 = 0.69$$

$$q(s_4) = \sum_{j=1}^{N} p(s_j) = 0.14 + 0.49 + 0.06 + 0.31 = 1$$



例如，从区间［0, 1］中产生4个随机数:

$r_1 = 0.450126,\quad r_2 = 0.110347,\quad r_3 = 0.572496,\quad r_4 = 0.98503$

染色体	适应度	选择概率	积累概率	选中次数
s_1=01101	169	0.14	0.14	1
s_2=11000	576	0.49	0.63	2
s_3=01000	64	0.06	0.69	0
s_4=10011	361	0.31	1.00	1

于是，经复制得群体：

s_1'=11000（24），s_2'=01101（13）

s_3'=11000（24），s_4'=10011（19）

被选中两次

交叉

设交叉率p_c=100%，即S_1中的全体染色体都参加交叉运算。

设s_1'与s_2'配对，s_3'与s_4'配对。

s_1' =11000（24），s_2' =01101（13）

s_3' =11000（24），s_4' =10011（19）

分别交换后两位基因，得新染色体：

s_1''=11001（25），s_2''=01100（12）

s_3''=11011（27），s_4''=10000（16）

变异

设变异率p_m=0.001。这样，群体S_1中共有

5×4×0.001=0.02 位基因可以变异。

0.02位显然不足1位，所以本轮遗传操作不做变异。

于是，得到第二代种群S_2：

s_1=11001（25），s_2=01100（12）

s_3=11011（27），s_4=10000（16）

第二代种群S_2中各染色体的情况

染色体	适应度	选择概率	积累概率	估计的选中次数
s_1=11001	625	0.36	0.36	1
s_2=01100	144	0.08	0.44	0
s_3=11011	729	0.41	0.85	2
s_4=10000	256	0.15	1.00	1

假设这一轮选择-复制操作中，种群S_2中的4个染色体都被选中，则得到群体：

s_1'=11001（25），s_2'= 01100（12）

s_3'=11011（27），s_4'= 10000（16）

做交叉运算，让s_1'与s_2'，s_3'与s_4'分别交换后三位基因，得

s_1''=11100（28），s_2'' = 01001（9）

s_3'' =11000（24），s_4'' = 10011（19）

这一轮仍然不会发生变异。

于是，得第三代种群S_3：

s_1=11100（28），s_2=01001（9）

s_3=11000（24），s_4=10011（19）

第三代种群S_3中各染色体的情况

染色体	适应度	选择概率	积累概率	估计的选中次数
s_1=11100	784	0.44	0.44	2
s_2=01001	81	0.04	0.48	0
s_3=11000	576	0.32	0.80	1
s_4=10011	361	0.20	1.00	1

被选中两次

设这一轮的选择-复制结果为：

s_1'=11100（28），s_2'=11100（28）

s_3'=11000（24），s_4'=10011（19）

做交叉运算，让s_1'与s_4'，s_2'与s_3'分别交换后两位基因，得

s_1''=11111（31），s_2''=11100（28）

s_3''=11000（24），s_4''=10000（16）

这一轮仍然不会发生变异。

于是，得第四代种群S_4：

s_1=11111（31），s_2=11100（28）

s_3=11000（24），s_4=10000（16）

出现了最优解

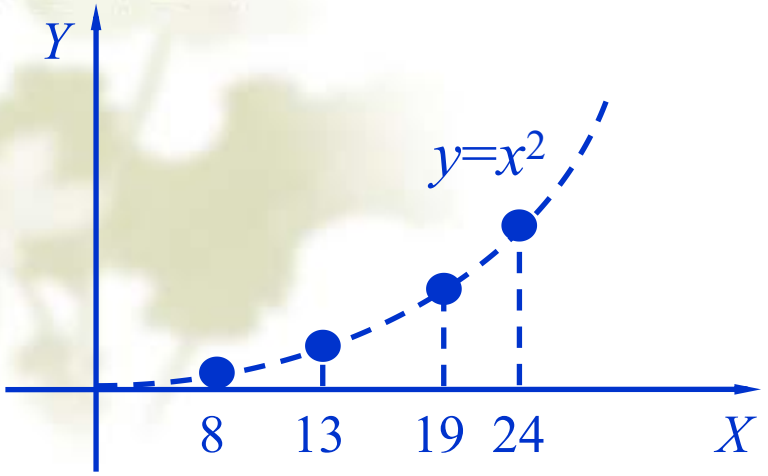
显然，在这一代种群中已经出现了适应度最高的染色体s_1=11111。于是，遗传操作终止，将染色体（11111）作为最终结果输出。

然后，将染色体“11111”解码为表现型，即得所求的最优解：31。

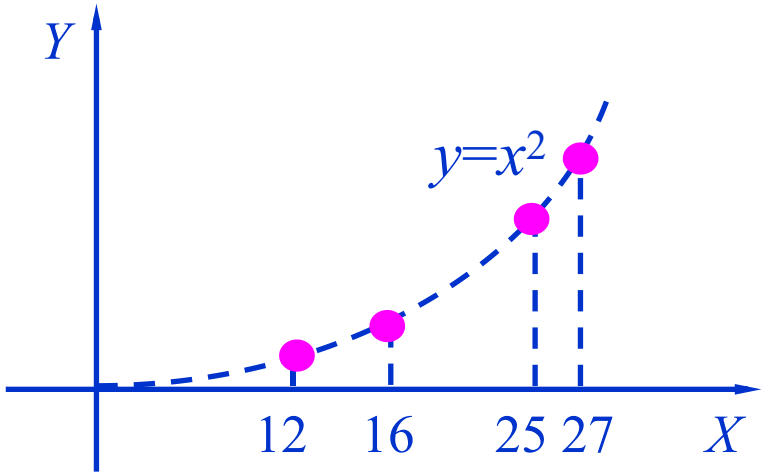
将31代入函数$y=x^2$中，即得原问题的解，即函数$y=x^2$的最大值为961。

例：应用遗传算法求函数 $f(x)=x^2$ 的最大值，$x \in [0,31]$

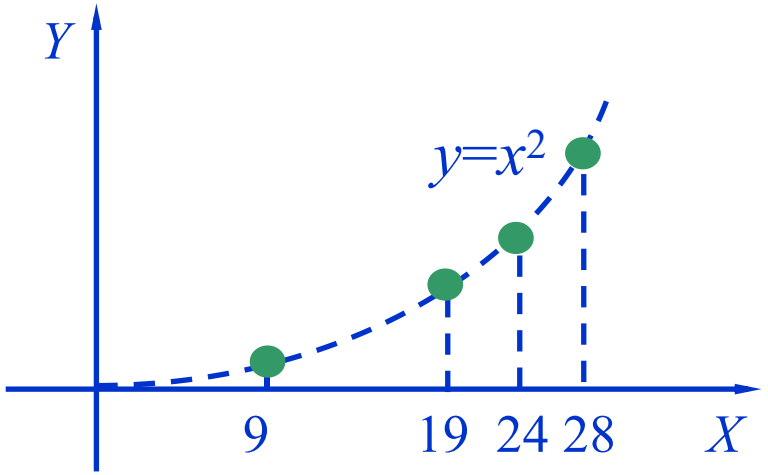
个体号	初始种群	X值	适应度	选择概率	适应读期望值	选择次数	配对池	交叉位置	配对	新一代种群	X值	适应度
1	01101	13	169	0.14	0.58	1	01101	0110|1	2	01100	12	144
2	11000	24	576	0.49	1.97	2	11000	1100|0	1	11001	25	625
3	01000	8	64	0.06	0.22	0	11000	11|000	4	11011	27	729
4	10011	19	361	0.31	1.23	1	10011	10|011	3	10000	16	256
适应度总和 $\sum f_i$			1170	1.0	4.00	4						1754
平均适应度 $\bar{f}=\sum f_i/n$			293	0.25	1.00	1.0						439
最大适应度			576	0.49	1.97	2.0						729



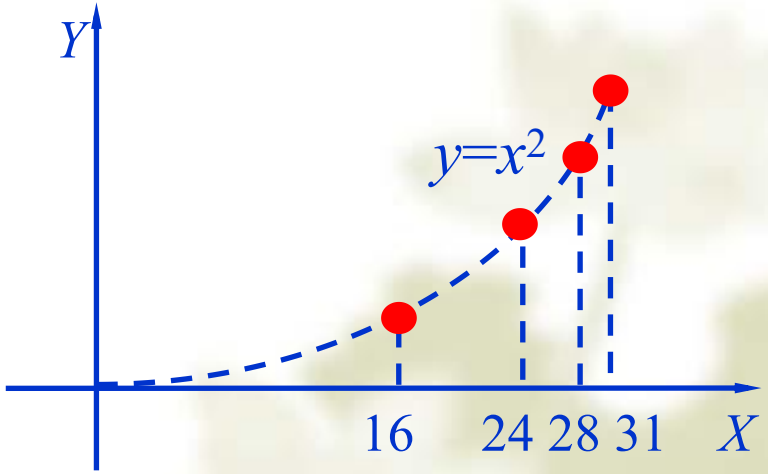
第一代种群及其适应度



第二代种群及其适应度



第三代种群及其适应度



第四代种群及其适应度

产生的四代种群的规律是：适应度的整体水平越来越好，直到最优解出现！

例2：求下述二元函数的最大值：

$$\max \quad f(x_1,x_2)=x_1^2+x_2^2$$
$$\text{s.t.} \quad x_1 \in \{1,2,3,4,5,6,7\}$$
$$x_2 \in \{1,2,3,4,5,6,7\}$$

(1) 个体编码

例如，基因型 X＝101110 所对应的表现型是：x＝[5，6]。

(2) 初始群体的产生

群体规模的大小取为4，即群体由4个个体组成，每个个体可通过随机方法产生。

如：011101，101011，011100，111001

(3) 适应度计算

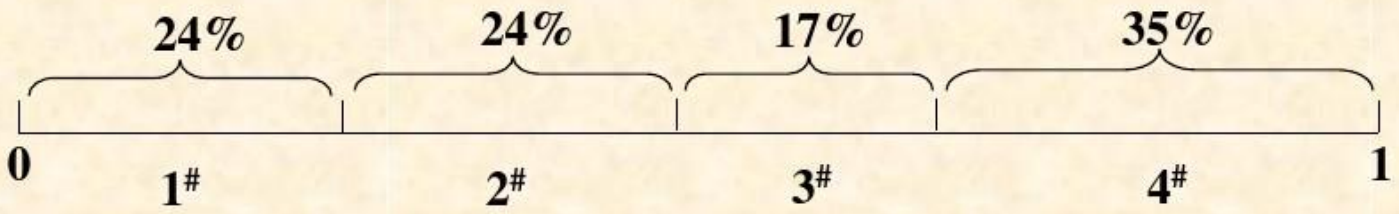
可直接利用目标函数值作为个体的适应度。

(4) 选择运算

本例中，我们采用与适应度成正比的概率来确定各个个体复制到下一代群体中的数量。其具体操作过程是：

- 先计算出群体中所有个体的适应度的总和 Σf_i (i=1.2,…,M);
- 其次计算出每个个体的相对适应度的大小 $f_i / \Sigma f_i$ ，它即为每个个体被遗传到下一代群体中的概率，
- 每个概率值组成一个区域，全部概率值之和为1；
- 最后再产生一个0到1之间的随机数，依据该随机数出现在上述哪一个概率区域内来确定各个个体被选中的次数。

个体编号	初始群体p(0)	x_1 x_2	适值	占总数的百分比	选择次数	选择结果
1	011101	3 5	34	0.24	1	011101
2	101011	5 3	34	0.24	1	111001
3	011100	3 4	25	0.17	0	101011
4	111001	7 1	50	0.35	2	111001
总和			143	1		



(5) 交叉运算

交叉运算是遗传算法中产生新个体的主要操作过程，它以某一概率相互交换某两个个体之间的部分染色体。

本例采用单点交叉的方法，其具体操作过程是：

- 先对群体进行随机配对；
- 其次随机设置交叉点位置；
- 最后再相互交换配对染色体之间的部分基因。

个体编号	选择结果	配对情况	交叉点位置	交叉结果
1 2 3 4	01 1101 11 1001 1010 11 1110 01	1-2 3-4	1-2：2 3-4：4	011001 111101 101001 111011

可以看出，其中新产生的个体“111101”、“111011”的适应度较原来两个个体

的适应度都要高。

(6) 变异运算

个体编号	交叉结果	变异点	变异结果	子代群体p(1)
1	011001	4	011101	011101
2	111101	5	111111	111111
3	101001	2	111001	111001
4	111011	6	111010	111010

对群体P(t)进行一轮选择、交叉、变异运算之后可得到新一代的群体p(t+1)。

个体编号	子群体p(1)	x_1 x_2	适值	占总数的百分比
1	011101	3 5	34	0.14
2	111111	7 7	98	0.42
3	111001	7 1	50	0.21
4	111010	7 2	53	0.23
总和			235	1

从上表中可以看出，群体经过一代进化之后，其适应度的最大值、平均值都得到了明显的改进。事实上，这里已经找到了最佳个体“111111”。

个体编号	初始群体p(0)	x_1 x_2	适值 $f_i(x_1,x_2)$		占总数的百分比 $f_i / \Sigma f$
1	011101	3 5	34	$\Sigma f_i=143$	0.24
2	101011	5 3	34	$f_{max}=50$	0.24
3	011100	3 4	25	$\bar{f}=35.75$	0.17
4	111001	7 1	50		0.35

选择次数	选择结果	配对情况	交叉点位置	交叉结果	变异点	变异结果
1	011101			011001	4	011101
1	111001	1-2	1-2：2	111101	5	111111
0	101011	3-4	3-4：4	101001	2	111001
2	111001			111011	6	111010

子代群体p(1)	x_1 x_2	适值 $f_i(x_1,x_2)$		占总数的百分比 $f_i / \Sigma f$
011101	3 5	34	$\Sigma f_i=253$	0.14
111111	7 7	98	$f_{max}=98$	0.42
111001	7 1	50	$\bar{f}=58.75$	0.21
111010	7 2	53		0.23

例 3. 用遗传算法求解TSP

TSP也称货郎担问题，是一个较古老的问题，最早可以追溯到1759年Euler提出的骑士旅行问题，1948年由美国兰德公司推动，TSP成为近代组合优化领域的一个典型问题。应该说，TSP是一个具有广泛应用背景和重要理论价值的组合优化问题，它已经被证明属于NP难题。几十年来，出现了很多近似优化算法，如近邻法、贪心算法、最近插入法、双极小生成树法等。此外还可以用Hopfield神经网络方法、模拟退火方法和遗传算法等方法。

已知n个城市之间的距离，现有一推销员必须访问这n个城市，并且每个城市只能访问一次，最后必须返回出发城市。如何安排他对这些城市的访问次序，可使其旅行路线的总长最短。

TSP搜索空间随着城市数n的增加而增大，所有的旅行路线组合数为(n-1)!/2。5个城市的情形对应12条路线，10个城市的情形对应181440条路线。例如，对于20城市的TSP城市坐标信息，应用启发式搜索A*算法，获得最优旅程长度为24.38，该最优解在展开17222个节点后找到。应用模拟退火得到同样的最优解，从随机初始旅程出发的1000次运行中，找到最优旅程的情况占79.2%。采用遗传算法，种群大小为300，交叉概率为0.45，终止代数300，共运行500次，得到同样的最优解。

总体思路：

将一个合法的城市序列$s=(c_1, c_2, \ldots, c_n, c_{n+1})$($c_{n+1}$就是$c_1$)作为一个个体。这个序列中相邻两城之间的距离之和的倒数就可作为相应个体s的适应度，从而适应度函数就是

$$f(s)=1/\sum_{i=1}^{n} d(c_i, c_{i+1})$$

用符号A、B、C、D、E代表相应的城市，用这5个符号的序列表示可能解即染色体。然后设计合适的染色体和相应的遗传运算，使得这些遗传运算对染色体集合封闭。

TSP问题的遗传算法求解

- 步骤一： 个体编码及种群初始化
- 步骤二： 适应度选择
- 步骤三： 交叉操作
- 步骤四： 变异操作
- 步骤五： 重复二、三、四步， 直至结束

令城市(点)数目为 N

步骤一： 初始化

- 个体编码

  取长度为N的数字串， 串中数字互不重复， 取值范围为[1,N]之间的整数。则每一个数字串代表一个个体， 个体中数字出现的位置表征路径中城市出现的顺序。

- 初始种群

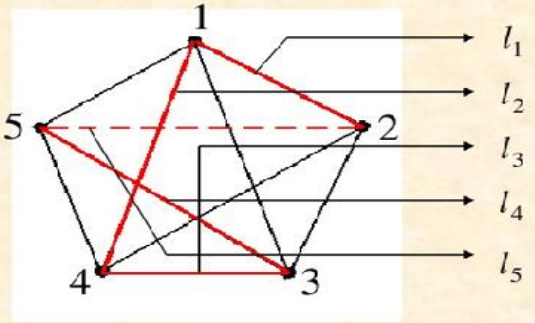
  令种群中有 M 个个体， 可随机产生 M 个数字串构成初始种群。 例如：
  将数字串 1234…N 上的数字进行随机的交换

步骤二：适应度选择

- 适应度的计算

对于个体 j，适应度为：

$$f_j = \frac{1}{\sum_{i=1}^{N} l_i}$$



被选中作为父个体的概率：

$$p_j = \frac{f_j}{\sum_{j=1}^{M} f_j} \qquad \sum p_j = 1$$

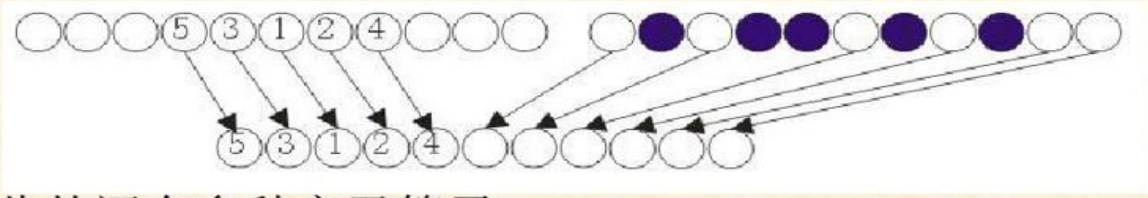
选择 M 次
重新生成种群

步骤三：交叉操作

- TSP中交叉算子的特点

要保证生成的解为有效解

从一个父个体中随机选取一段子串A，在另一个父个体中将A中出现的数字去掉形成串B，AB为一个子串



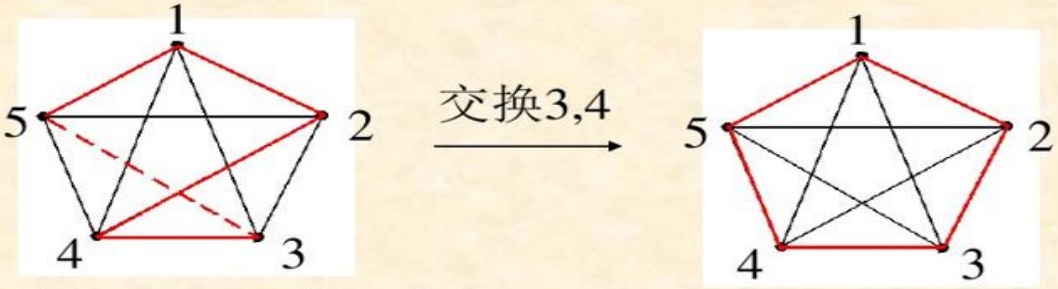
此外还有多种交叉算子

步骤四：变异操作

- 常用的变异操作：

  随机选取两个相邻位置的数字，交换其顺序。

  51243(5) → 51234(5)



此外还有多种变异算子

步骤五：

- 反复执行步骤二、三、四
- 结束判定

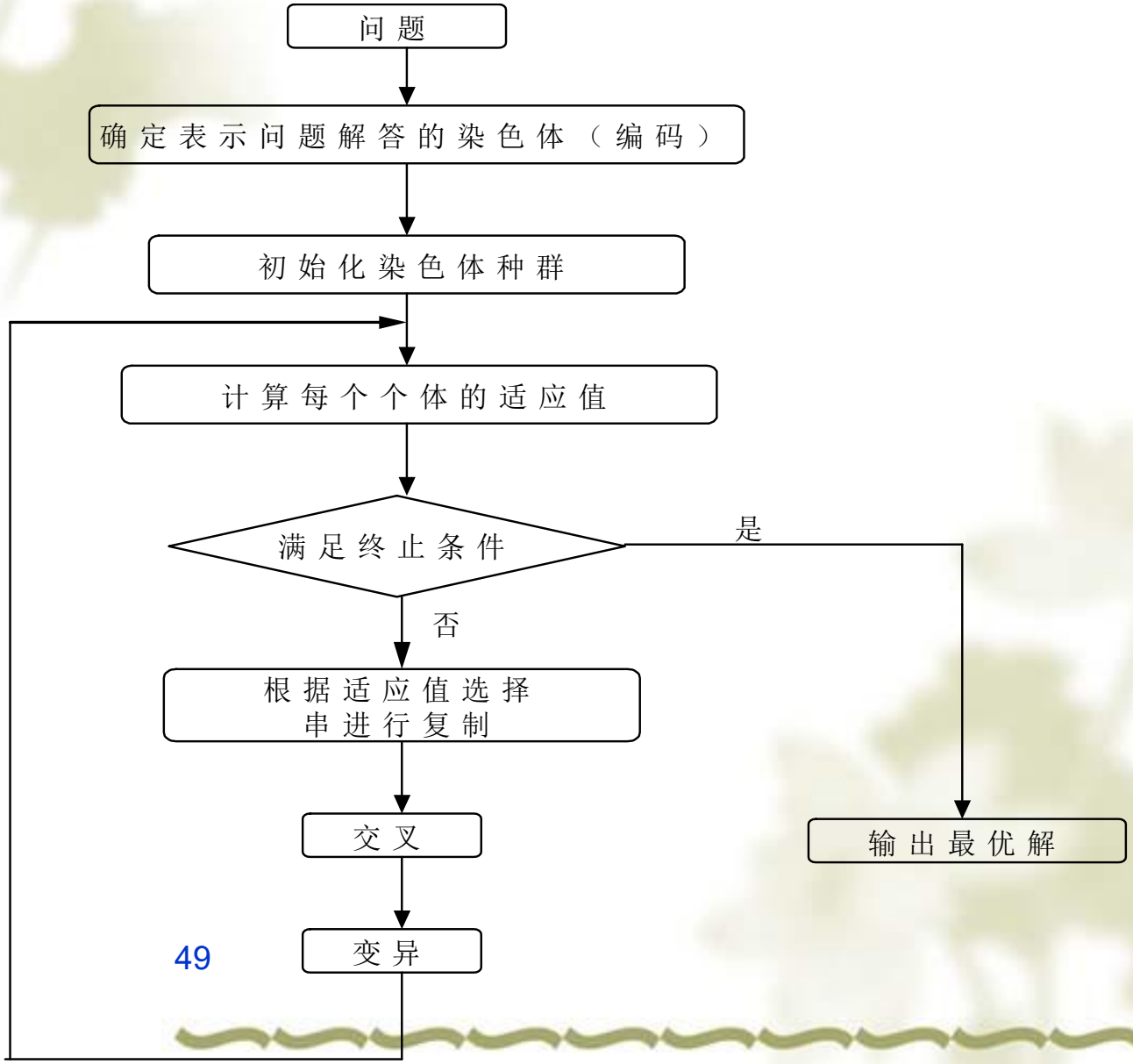
  循环执行 G 次 (例如 G=500) 后

  当最优个体的总路径长度小于预期时

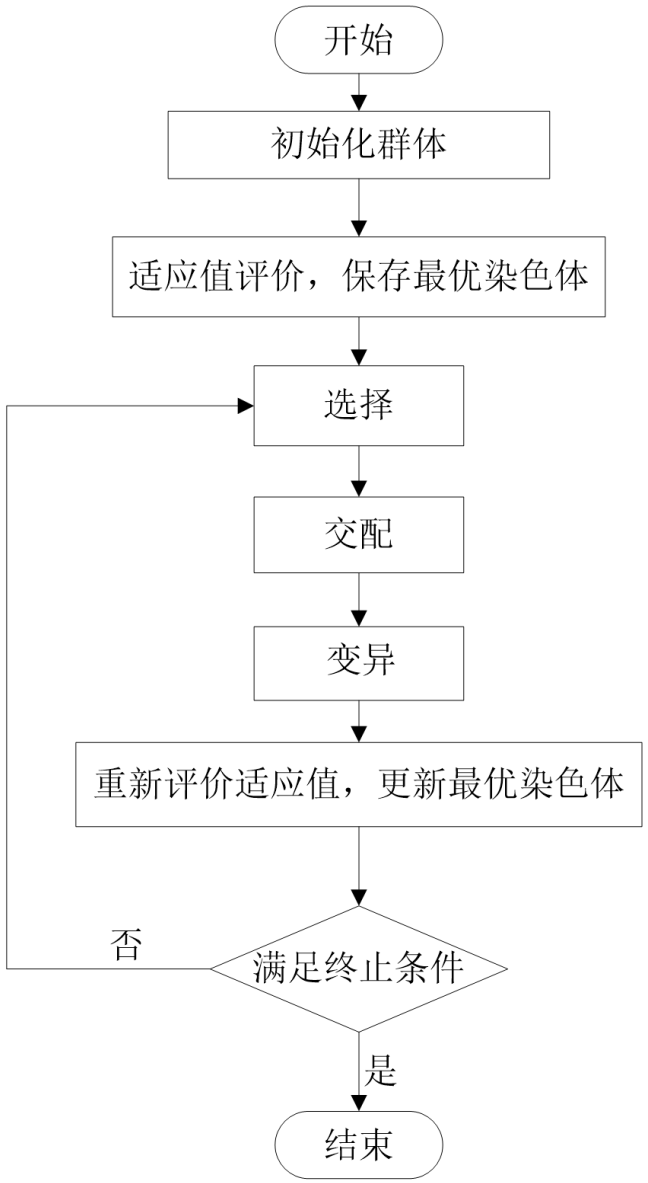
3.5 基本遗传算法SGA

- 遗传算法是一种群体型操作，该操作以群体中的所有个体为对象。遗传算法就是对种群中的染色体反复进行三种遗传操作，使其朝着适应度增高的方向不断更新换代，直至出现了适应度满足目标条件的染色体为止。
- 标准遗传算法(Simple Genetic Algorithms,简称SGA)又叫基本遗传算法或简单遗传算法。
- 是一种统一的最基本的遗传算法，它只使用选择、交叉、变异这三种基本遗传算子，其遗传进化操作过程简单，容易理解，是其他遗传算法的雏形和基础，它不仅给各种遗传算法提供了一个基本框架。

基本遗传算法SGA流程框架



遗传算法流程图和伪代码




```
/*
   P(t)表示某一代的群体，t为当前进化代数
   Best 表示目前已找到的最优解
*/
Procedure GA
begin
      t←0;
      initialize(P(t));                    //初始化群体
      evaluate(P(t));                      //适应值评价
      keep_best(P(t));                  //保存最优染色体
      while (不满足终止条件) do
      begin
          P(t)← selection(P(t));          //选择算子
          P(t)← crossover(P(t));          //交配算子
          P(t)← mutation(P(t));           //变异算子
          t←t+1;
          P(t)←P(t-1);
          evaluate(P(t));
          if(P(t)的最优适应值大于Best的适应值)
            //以P(t)的最优染色体替代Best
            replace(Best);
          end if
       end
end
```

基本遗传算法SGA

步1 在搜索空间U上定义一个适应度函数$f(x)$，给定种群规模N，交叉率P_c和变异率P_m，代数T；

步2 随机产生U中的N个个体$s_1, s_2, \cdots, s_N$，组成初始种群$S=\{s_1, s_2, \cdots, s_N\}$，置代数计数器$t=1$；

步3 计算S中每个个体的适应度f()；

步4 若终止条件满足，则S中适应度最大的个体即为所求结果，算法结束。

步5 按选择概率$P(x_i)$所决定的选中机会，每次从S中随机选定1个个体并将其染色体复制，共做N次，然后将复制所得的N个染色体组成群体S_1；

步6 按交叉率P_c所决定的参加交叉的染色体数c，从S_1中随机确定c个染色体，配对进行交叉操作，并用产生的新染色体代替原染色体，得群体S_2；

步7 按变异率P_m所决定的变异次数m，从S_2中随机确定m个染色体，分别进行变异操作，并用产生的新染色体代替原染色体，得群体S_3；

步8 将群体S_3作为新一代种群，即用S_3代替S，$t = t+1$，转步3；

基本遗传算法的形式化定义

可定义为一个8元组：

$$SGA=(C,E,P_0,M,\Phi,\Gamma,\Psi,T)$$

C---个体的编码方法；

E---个体适应度评价函数；

Φ---选择算子；

Ψ ---变异算子

P_0---初始群体；

M---群体大小；

Γ---交叉算子；

T---遗传运算终止条件。

算法的终止条件有以下几种：

- 进化代数限制；
- 适应度已经达到饱和，继续进化不会造成适应度更好的个体；
- 计算耗费的资源限制（例如计算时间、计算占用的内存等）；
- 一个个体已经满足最小值的条件，即最小值已经找到；
- 人为干预；或者以上两种或更多种的组合。

一般地，最简单的算法终止条件有如下二种：

- **完成了预先给定的进化代数则停止；**
- **群体中的最优个体在连续若干代没有改进或平均适应度在连续若干代基本没有改进时停止。**

3.6 遗传算法的特点与应用

遗传算法的基本思想：

在求解问题时从多个解开始，然后“生成-测试”式地不断迭代以产生新的解。

遗传算法是一种迭代式、*随机*、*全局*、*优化*、*搜索算法*。

回顾遗传算法执行过程。首先，在解空间中取一群点作为遗传开始的第一代(算法随机生成一定数量的个体，操作者对这个随机产生过程可以进行干预，播下已经部分优化的种子)。之后一代一代的进化，在进化过程中的每一代，每一个个体的适应程度都被评价，通过自然选择和变异产生下一代个体并组成群落。对于一个优化问题，一定数量的候选解（被视为个体）被表示为抽象的字符串或数字串，叫做染色体或者基因串。通过进化向更好的解发展。

（1）随机性体现在：

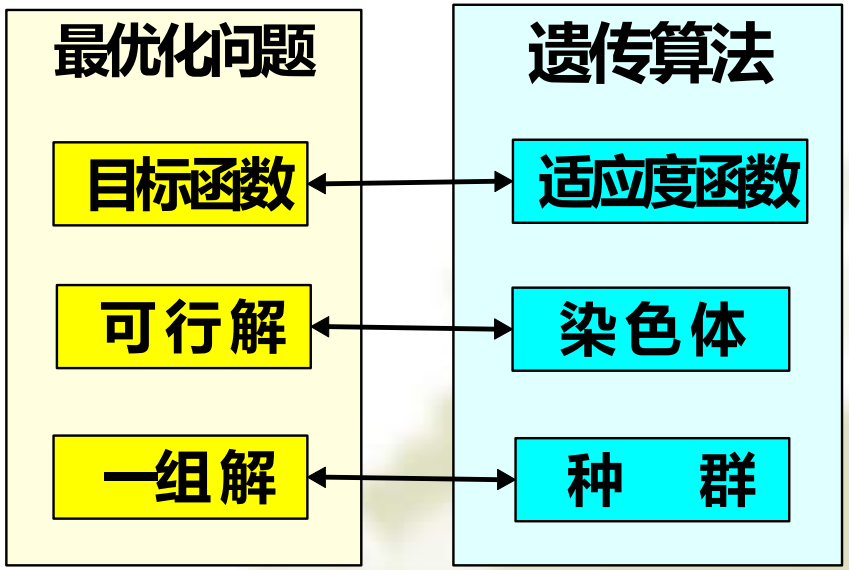
——遗传算法的搜索随机地始于搜索空间的一个点集，而不像图搜索那样固定地始于搜索空间的初始节点或终止节点，所以遗传算法是一种随机搜索算法。

——此外，遗传算法利用选择、交叉、变异等算子进行随机操作而不是利用确定性规则。

——遗传算法不是采用确定性规则，而是采用概率作为一种工具来引导其搜索过程朝着搜索空间的更优化的解区域移动。因此虽然看起来它是一种盲目搜索方法，但实际上有明确的搜索方向。

（2）优化算法：

遗传算法总是在寻找优解，而不像图搜索那样并非总是要求优解，而一般是设法尽快找到解，所以遗传算法又是一种优化搜索算法。遗传算法非常适合求解组合优化问题。



（3）遗传算法一般是直接在解空间搜索，而不像图搜索那样一般是在问题空间搜索，最后才找到解。

（4）遗传算法从问题解的串集开始搜索，而不是从单个解开始，这是遗传算法与传统优化算法的极大区别。

许多传统优化搜索方法都是单点搜索算法，问题的解从搜索空间中的当前解(点)移到另一解(点)。这种点对点的搜索方法，对于多峰分布的搜索空间常常会陷于局部的某个单峰的优解。

相反，遗传算法是采用同时处理群体中多个个体的方法，即同时对搜索空间中的多个解进行评估。更形象地说，遗传算法是在并行地爬多个峰。

（5）遗传算法的处理对象不是参数本身，而是对参数集进行了编码的个体。遗传算法提供了一种求解复杂系统优化问题的通用框架，它不依赖于问题的具体领域，对问题的种类有很强的鲁棒性，因而简单易懂、通用性强。

（6）遗传算法的适应性强，除需知适应度函数外，几乎不需要其他的先验知识。适应度函数不仅不受连续可微的约束，而且其定义域可以任意设定。这一特点使得遗传算法的应用范围大大扩展。

（7）遗传算法的搜索过程是从空间的一个点集(种群)到另一个点集(种群)的搜索，因而它实际是一种并行搜索，适合大规模并行计算。

（8）遗传算法长于全局搜索，它不受搜索空间的限制性假设的约束，不要求连续性，这种搜索减少了陷入局部最优解的风险，使遗传算法具有较好的全局搜索性能。

1. 遗传算法的特点

优点：

1. 搜索从群体出发，具有潜在的并行性；
2. 解决问题的方法具有普适性，能解决的问题范围也很广；
3. 搜索使用评价函数（适应度函数）启发，过程简单；
4. 全局收敛性。
5. 具有可扩展性，容易与其他算法（粒子群、模拟退火等）结合。

不足：

1. 算法参数的选择严重影响解的品质，而目前这些参数的选择大部分是依靠经验；
2. 遗传算法的本质是随机性搜索，不能保证所得解为全局最优解；
3. 在处理具有多个最优解的多峰问题时容易陷入局部最小值而停止搜索，造成早熟问题，无法达到全局最优。

2. 遗传算法的应用

（1）函数优化

函数优化是遗传算法的经典应用领域，也是遗传算法进行性能评价的常用算例。尤其是对非线性、多模型、多目标的函数优化问题，采用其他优化方法较难求解，而遗传算法却可以得到较好的结果。

（2）组合优化

随着问题的增大，组合优化问题的搜索空间也急剧扩大，采用传统的优化方法很难得到最优解。遗传算法是寻求这种满意解的最佳工具。例如遗传算法已经在求解旅行商问题、背包问题、装箱问题、图形划分问题等方面得到成功的应用。

（3）自动控制

在自动控制领域中有很多与优化相关的问题需要求解，遗传算法已经在其中得到了初步的应用。例如，利用遗传算法进行控制器参数的优化、基于遗传算法的模糊控制规则的学习、基于遗传算法的参数辨识、基于遗传算法的神经网络结构的优化和权值学习等。

（4）图像处理

遗传算法可用于图像处理过程中的扫描、特征提取、图像分割等的优化计算。目前遗传算法已经在模式识别、图像恢复、图像边缘特征提取等方面得到了应用。

（5）规划、设计（如通信网络设计）、调度（如作业车间调度、生产调度、运输问题）、配置问题

在很多情况下，采用建立数学模型的方法难以对生产调度问题进行精确求解。在现实生产中多采用一些经验进行调度。遗传算法是解决复杂调度问题的有效工具，在单件生产车间调度、流水线生产车间调度、生产规划、任务分配等方面遗传算法都得到了有效的应用。

（6）机器人

例如，遗传算法已在移动机器人路径规划、关节机器人运动轨迹规划、机器人结构优化和行为协调等方面得到研究和应用。

（7）人工生命

用计算机、机械等人工媒体模拟或构造出的具有生物系统特有行为的人造系统。人工生命与遗传算法有着密切的联系，遗传算法为人工生命的研究提供了一个有效的工具。

（8）机器学习

基于遗传算法的机器学习在很多领域都得到了应用。例如，采用遗传算法实现模糊控制规则的优化，可以改进模糊系统的性能；遗传算法可用于神经网络连接权的调整和结构的优化；采用遗传算法设计的分类器系统可用于学习式多机器人路径规划。

（9）数据挖掘

演化算法（evolutionary algorithm）

- 演化算法（Evolutionary Algorithms）是指一大类受自然演化启发的启发式随机优化算法，通过“突变重组”和“自然选择”这两种机制来模拟自然演化过程。演化算法有很多种实现方法，如遗传算法(genetic algorithm)、遗传规划(genetic programming，GP)、演化策略(evolutionary strategy)等。
- 遗传算法（Genetic Algorithm, GA）是演化学习的代表性算法之一，其受到达尔文“物竞天择、适者生存”进化论思想启示，引入选择(selection)、交叉(crossover)和变异(mutation)等操作以搜索方式来进行优化求解。遗传算法由密歇根大学的约翰·霍兰德（John H. Holland）和他的同事于二十世纪六十年代在对细胞自动机（cellular automata）进行研究时率先提出。
- 在遗传算法中，对于一个待求解的最优化问题，先给定一定数量候选解（称为个体或染色体），然后让这些染色体所构成种群向最优解进化。在每一次进化过程中，计算每个染色体（即候选解）的适应度（fitness），根据适应度选择合适染色体，随后施以选择和变异来产生新的生命种群，该种群在算法下一次迭代中被作为当前种群。

演化算法（evolutionary algorithm）

步骤1：初始化具有若干规模数目的群体。当前进化代数Generation=0

步骤2：采用评估函数计算群体中每个染色体的适应值，保存适应值最大的染色体Best。

步骤3：采用轮盘赌选择算法对群体中染色体进行选择，产生同样规模的种群。

步骤4：按照概率从种群中选择两对染色体进行交叉操作，即对所选中两对染色体中相应基因片段信息进行交换。交叉所得染色体进入新种群，未交叉操作染色体被复制进入新种群。

步骤5：按照概率对新种群中被选中染色体的若干基因片段进行变异操作。变异后染色体取代原有染色体进入新种群，未发生变异染色体直接进入新群体。

步骤6：计算种群中各个染色体的适应值。倘若种群中染色体最大适应值大于已知Best适应值，则取代原有Best。

步骤7：当前进化代数Generation加1。如果Generation超过规定的最大进化代数或Best达到规定的误差要求，算法结束；否则返回步骤3。

例5. 已知函数$y=f(x_1,x_2,x_3,x_4)=\frac{1}{x_1^2+x_2^2+x_3^2+x_4^2+8}$，其中$-5\le x_1,x_2,x_3,x_4\le 5$，用遗传算法求解函数$y$的最大值及对应变量$x_1$、$x_2$、$x_3$和$x_4$的取值。

步骤1：初始化

假设种群规模数目为5，使用浮点数编码方式构造染色体（即候选解）。每个染色体以四维向量形式表示，对应函数$f(x_1,x_2,x_3,x_4)$中四个变量取值。初始化后得到如下五个染色体：

$$C_1=(-2.1351,2.0917,-0.1327,-4.1006)$$

$$C_2=(1.0152,-3.9811,-2.6638,3.7535)$$

$$C_3=(4.0589,2.1904,-0.1503,0.0023)$$

$$C_4=(-3.4098,-3.0714,-0.9008,-4.3712)$$

$$C_5=(0.2073,2.9932,-4.0802,1.8794)$$

步骤2：计算适应值

如下计算每个染色体适应值 $Eval(C) = f(x_1, x_2, x_3, x_4) = \frac{1}{x_1^2+x_2^2+x_3^2+x_4^2+8}$：

$$Eval(C_1) = f(-2.1351, 2.0917, -0.1327, -4.1006) = 0.0296152$$

$$Eval(C_2) = f(1.0152, -3.9811, -2.6638, 3.7535) = 0.0217087$$

$$Eval(C_3) = f(4.0589, 2.1904, -0.1503, 0.0023) = 0.0341354$$

$$Eval(C_4) = f(-3.4098, -3.0714, -0.9008, -4.3712) = 0.0204169$$

$$Eval(C_5) = f(0.2073, 2.9932, -4.0802, 1.8794) = 0.0268944$$

因此 $Best = C_3, Eval(Best) = 0.0341354$

步骤3：选择

采用轮盘赌选择算法对染色体进行选择，得到新种群为：

$$C_1' = (1.0152, -3.9811, -2.6638, 3.7535)$$

$$C_2' = (4.0589, 2.1904, -0.1503, 0.0023)$$

$$C_3' = (1.0152, -3.9811, -2.6638, 3.7535)$$

$$C_4' = (1.0152, -3.9811, -2.6638, 3.7535)$$

$$C_5' = (0.2073, 2.9932, -4.0802, 1.8794)$$

步骤4：交叉

具体而言，随机生成1至4之间一个自然数作为交叉位，每对染色体交换交叉位及之后基因信息（例如交叉位为3，则交换这对染色体3号位以及之后的基因信息）：

C_1'和C_2'交叉位为3，交换这对染色体3号位以及之后基因信息，得到新的染色体：

$$C_1'' = (1.0152, -3.9811, -0.1503, 0.0023)$$

$$C_2'' = (4.0589, 2.1904, -2.6638, 3.7535)$$

C_4'和C_5'交叉位为4，交换这对染色体4号位以及之后基因信息，得到新的染色体：

$$C_4'' = (1.0152, -3.9811, -2.6638, 1.8794)$$

$$C_5'' = (0.2073, 2.9932, -4.0802, 3.7535)$$

步骤5：变异

假设变异概率为0.1。对于每个染色体中若干基因随机生成[0,1]的随机数，若该随机数小于0.1，则改变该基因信息取值，否则不改变该基因信息取值：

C_3''的一号位基因采样所得随机数小于0.1，该基因发生随机变异，其它基因信息保持不变：

$$C_3'' = (1.0152, -3.9811, -2.6638, 3.7535) \rightarrow (3.0953, -3.9811, -2.6638, 3.7535)$$

C_4''的二号位基因采样所得随机数小于0.1，该基因发生随机变异，其它基因信息保持不变：

$$C_4'' = (1.0152, -3.9811, -2.6638, 1.8794) \rightarrow (1.0152, 0.0153, -2.6638, 1.8794)$$

步骤6：重新评价染色体适应值，更新*Best*

步骤7：判断结束

如果符合算法终止条件，则输出所找到最优解*Best*，退出程序，否则返回步骤3继续执行。

3.7 遗传算法机理的深入探讨

遗传算法的五个基本要素：

- 参数编码
- 初始群体的设定
- 适应度函数的设计
- 控制参数设定
- 遗传操作设计

一、参数编码

1、 位串编码

一维染色体编码方法：将问题空间的参数编码为一维排列的染色体的方法。

（1） 二进制编码

二进制编码：用若干二进制数表示一个个体，将原问题的解空间映射到位串空间 $B=\{0, 1\}$上，然后在位串空间上进行遗传操作。

例4. 求下列一元函数的最大值：

$$f(x) = x \cdot \sin(10\pi \cdot x) + 1.0$$

其中$x \in [-1, 2]$，求解结果精确到6位小数。

（1）二进制编码要求取多少位？

- 区间长度为3，求解结果精确到6位小数，因此可将自变量定义区间划分为3×10^6等份。
- 又因为$2^{21} < 3\times10^6 < 2^{22}$，所以二进制编码长度至少需要22位
- 编码过程实质是将区间[-1，2]内对应的实数值转化为一个二进制串（$b_{21}b_{20}b_{19}b_{18}...b_1b_0$）。

(1) 将二进制串 $(b_{21}\,b_{20}\,b_{19}\,\cdots b_0)$ 转换为十进制为

$$x' = (b_{21}\,b_{20}\,b_{19}\,\cdots b_0)_2 = (\sum_{i=0}^{21} b_i \cdot 2^i)_{10}$$

(2) 二进制串对应的实数 x 为

$$x = -1.0 + \frac{x'}{2^{22}-1} \times 3$$

例如 (100010111 0110101000 111) 表示 0.637197, 因为

$$x' = (\sum_{i=0}^{21} b_i \cdot 2^i)_{10} = 2288967$$

$$x = -1.0 + \frac{2288967}{2^{22}-1} \times 3 = 0.637197$$

显然 : (0000000000 0000000000 0) 表示 -1

(111111111 1111111111 11) 表示 2

二进制编码方法如下：

设某一参数的取值范围是[A，B]，用长度为r的二进制编码串来表示该参数，将[A，B]等分为2^r-1个子部分，记每个等分的长度为£，则它能产生2^r种不同的编码，其中

$$£=（B-A）/（2^r-1）$$

假设某一个体的编码是：X：$x_r x_{r-1} x_{r-2} \cdots x_2 x_1$

则上述编码对应的解码公式为

$$X=A+（B-A）/（2^r-1）* \sum x_i 2^{i-1}$$

- **优点　算法易于用生物遗传理论来解释并使得遗传操作（如杂交、变异等）很容易实现。**
- **缺点**
  - 相邻整数的二进制编码可能具有较大的Hamming距离
  - 二进制编码时，一般要先给出求解的精度以确定串长
  - 在求解高维优化问题时，二进制编码串将非常长 。

（2） Gray 编码

Gray编码:将二进制编码通过一个变换进行转换得到的编码。

二进制串 $< \beta_1 \beta_2 \ldots \beta_n >$ Gray $< \gamma_1 \gamma_2 \ldots \gamma_n >$

二进制编码 → Gray编码

$$\gamma_k = \begin{cases} \beta_1 & k = 1 \\ \beta_{k-1} \oplus \beta_k & k > 1 \end{cases}$$

Gray编码 → 二进制编码

$$\beta_k = \sum_{i=1}^{k} \gamma_i \pmod 2$$

2、实数编码

- 采用实数表达法不必进行数制转换，可直接在解的表现型上进行遗传操作。
- 多参数映射编码的基本思想：把每个参数先进行二进制编码得到子串，再把这些子串连成一个完整的染色体。
- 多参数映射编码中的每个子串对应各自的编码参数，所以，可以有不同的串长度和参数的取值范围。

3、有序串编码

- 有序问题：目标函数的值不仅与表示解的字符串的值有关，而且与其所在字符串的位置有关。
  - 需要针对具体问题专门设计有效且能保证后代合法的遗传算子。
  - 这类编码方案较多地使用在组合优化问题之中。

4、结构式编码

Goldberg等提出*MessyGA(mGA)*的遗传算法编码方法。

- 将问题的解表示树或图的形式的编码称为结构式编码。
- 因为遗传算子是直接在解的表现型上进行操作，这样使得我们比较容易加入与领域有关的知识和一些启发式的信息。

二、 群体设定

1. 初始种群的产生

（1）根据问题固有知识，把握最优解所占空间在整个问题空间中的分布范围，然后，在此分布范围内设定初始群体。

（2）随机产生一定数目的个体，从中挑选最好的个体加到初始群体中。这种过程不断迭代，直到初始群体中个体数目达到了预先确定的规模。

2. 种群规模的确定

- 群体规模太小，遗传算法的优化性能不太好，易陷入局部最优解。
- 群体规模太大，计算复杂。

三、适应函数的设计

个体的适应值即是它繁殖的能力，它将直接关系到其后代的数量，在遗传算法中，适应函数是用来区分群体中个体好坏的标准，是算法演化过程的驱动力，同时也是进行自然选择的唯一依据。

1. 原始适应函数

- 原始适应函数是问题求解目标的直接表示，通常采用问题的目标函数作为个体的适应度量 。
- 定义原始适应函数的方法可能不止一种，选择时要尽量反映问题本身整体特性，而不能只追求片面的目标 。

2. 标准适应函数

- 适应值会出现两种情形，一是极小情形即原始适应值越小个体性能越好；另一种是极大化情形即原始适应值越大个体性能越好 。
- 遗传算法中的某些选择策略则要求适应函数是非负的，而且适应值越大表明个体的性能越好。
- 对于极小化情形，标准适应值可定义为：

$$f_{Normal}(x) = f_{\max} - f(x)$$

3. 适应值的调节

- 存在问题：**过早收敛、停滞现象**
- 改变算法性能的方法之一是对适应值进行调节，即**通过变换，改变原适应值的比例关系**。

1. 将目标函数映射成适应度函数的方法

- 若目标函数为**最大化**问题，则 $Fit(f(x)) = f(x)$
- 若目标函数为**最小化**问题，则 $Fit(f(x)) = \dfrac{1}{f(x)}$

将目标函数转换为求最大值的形式,且保证函数值非负！

- 若目标函数为**最大化**问题，则

$$Fit(f(x)) = \begin{cases} f(x) - C_{\min} & f(x) > C_{\min} \\ 0 & \text{其他情况} \end{cases}$$

- 若目标函数为**最小化**问题，则

$$Fit(f(x)) = \begin{cases} C_{\max} - f(x) & f(x) < C_{\max} \\ 0 & \text{其他情况} \end{cases}$$

1. 将目标函数映射成适应度函数的方法(续)

存在界限值预选估计困难或者不能精确估计的问题!

- 若目标函数为**最大化**问题，则

$$Fit(f(x)) = \frac{1}{1 + c + f(x)} \qquad c \geq 0,\ c + f(x) \geq 0$$

- 若目标函数为**最小化**问题，则

$$Fit(f(x)) = \frac{1}{1 + c - f(x)} \qquad c \geq 0,\ c - f(x) \geq 0$$

- c ： 目标函数界限的保守估计值。

四、控制参数的设定

- **种群规模M**

  即群体中所含个体数目，一般取为20~100；

- **最大换代数T**

  遗传运算的终止进化代数，一般取为100-500

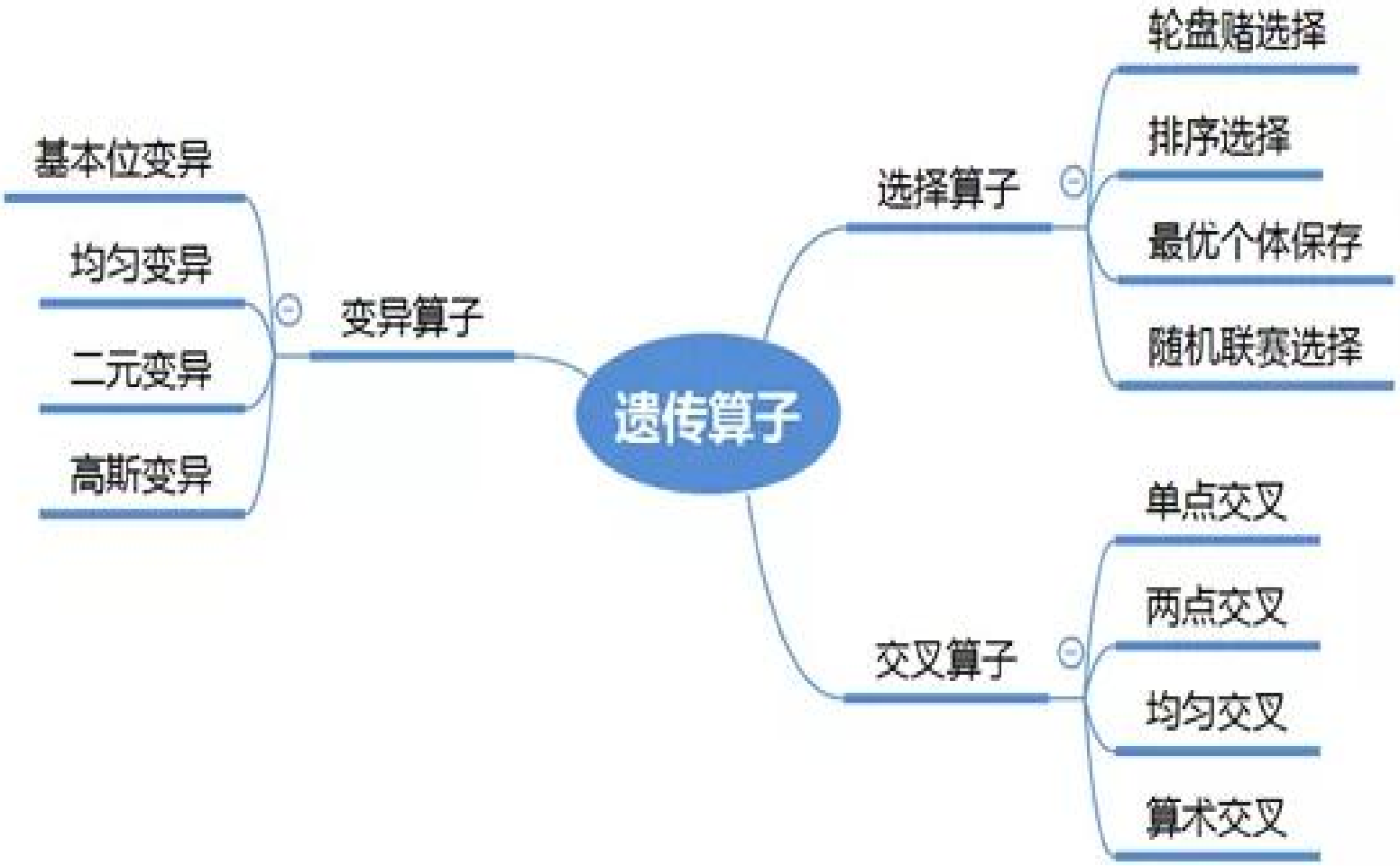
- **交叉率(crossover rate) P_c**

  参加交叉运算的染色体个数占全体染色体总数的比例，取值范围一般为0.5～0.99。

- **变异率(mutation rate) P_m**

  发生变异的基因位数所占全体染色体的基因总位数的比例，取值范围一般为0.0001～0.1。

五、遗传算子的设计


遗传算子
选择算子
轮盘赌选择
排序选择
最优个体保存
随机联赛选择
交叉算子
单点交叉
两点交叉
均匀交叉
算术交叉
变异算子
基本位变异
均匀变异
二元变异
高斯变异

1. 选择策略

- 选择操作也称为复制（reproduction）操作：从当前群体中按照一定概率选出优良的个体，使它们有机会作为父代繁殖下一代子孙。

- 判断个体优良与否的准则是各个个体的适应度值：个体适应度越高，其被选择的机会就越多。

- 不同的选择策略将导致不同的选择压力，即下一代中父代个体的复制数目的不同分配关系。

1、个体选择概率分配方法

（1）适应度比例方法（fitness proportional model）或蒙特卡罗法（Monte Carlo）

- 各个个体被选择的概率和其适应度值成比例。

- 个体 i 被选择的概率为：

$$p_{si} = \frac{f_i}{\sum_{i=1}^{M} f_i}$$

（2） 排序方法 （rank-based model）

首先根据个体 i 的适应值在群体中的排名来分配其选择概率P_i 。根据这个概率使用转盘选择

① 线性排序：J. E. Baker

- 群体成员按适应值大小从好到坏依次排列：$x_1, x_2, \cdots, x_N$
- 个体 x_i分配选择概率 p_i $\quad p_i = \dfrac{a - bi}{M(M+1)}$
- 按转盘式选择的方式选择父体

② 非线性排序： Z. Michalewicz

- 将群体成员按适应值从好到坏依次排列，并按下式非线性函数分配选择概率：

$$p_i = \begin{cases} q(1-q)^{i-1} & i = 1,2,\cdots,M-1 \\ (1-q)^{M-1} & i = M \end{cases}$$

- 可用其他非线性函数来分配选择概率，只要满足以下条件：

(1) 若 $P = \{x_1, x_2 \cdots, x_M\}$ 且 $f(x_1) \geq f(x_2) \geq \ldots \geq f(x_M)$, 则$p_i$满足 $p_1 \geq p_2 \geq \cdots \geq p_M$

(2) $\sum_{i=1}^{M} p_i = 1$

2、选择个体方法

- 选择算法：
  - 轮盘赌选择（roulette wheel selection）
  - 随机遍历抽样（stochastic universal selection）
  - 局部选择（local selection）
  - 截断选择（truncation selection）
  - 锦标赛选择（tournament selection）

（1）轮盘赌选择（roulette wheel selection）

- 按个体的选择概率产生一个轮盘，轮盘每个区的角度与个体的选择概率成比例。
- 产生一个随机数，它落入转盘的哪个区域就选择相应的个体交叉。

个体	1	2	3	4	5	6	7	8	9	10	11
适应度	2.0	1.8	1.6	1.4	1.2	1.0	0.8	0.6	0.4	0.2	0.1
选择概率	0.18	0.16	0.15	0.13	0.11	0.09	0.07	0.06	0.03	0.02	0.0
累积概率	0.18	0.34	0.49	0.62	0.73	0.82	0.89	0.95	0.98	1.00	1.00

第1轮产生一个随机数：**0.81**　　第2轮产生一个随机数：**0.32**

（2）最佳个体保存方法

- **最佳个体（elitist model）保存方法**：把群体中适应度最高的个体不进行交叉而直接复制到下一代中，保证遗传算法终止时得到的最后结果一定是历代出现过的最高适应度的个体。

（3）锦标赛选择方法（tournament selection model）

- **锦标赛选择方法**：从群体中随机选择个个体，将其中适应度最高的个体保存到下一代。这一过程反复执行，直到保存到下一代的个体数达到预先设定的数量为止。

- **随机竞争方法**（stochastic tournament）：每次按赌轮选择方法选取一对个体，然后让这两个个体进行竞争，适应度高者获胜。如此反复，直到选满为止。

（4）Boltzmann锦标赛选择

- 随机选取两个个体x_1, x_2，若$|f(x_1)-f(x_2)|\geq\theta$则选择适应值好的作为胜者，否则计算概率$p=\exp[-|f(x_1)-f(x_2)|/T]$，若$p>random[0,1)$选择差解，否则选择好解。

五、遗传算子的设计

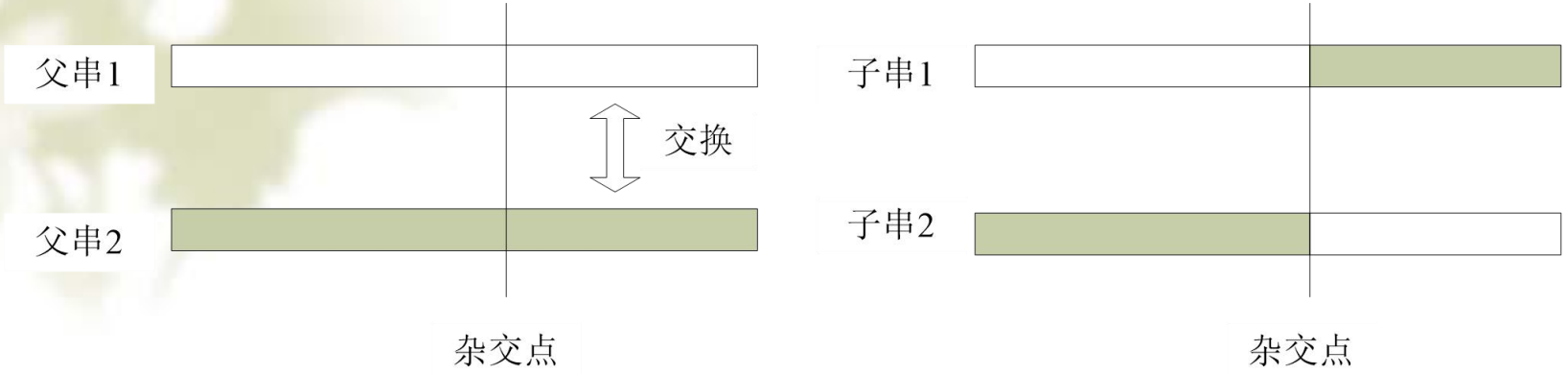
2.交叉算子

- 交叉运算，是指对两个相互配对的染色体依据交叉概率 Pc 按某种方式相互交换其部分基因，从而形成两个新的个体。
- 交叉运算是遗传算法区别于其他进化算法的重要特征，它在遗传算法中起关键作用，是产生新个体的主要方法。基本遗传算法中交叉算子采用单点交叉算子。

① 单点交叉

单点交叉又称为简单交叉，它是指在个体编码串中只随机设计一个交叉点，然后在该点相互交换两个配对个体的部分染色体。

单点交叉运算



交叉点

交叉前：

01000|0111000000010000

11100|0000011111000101

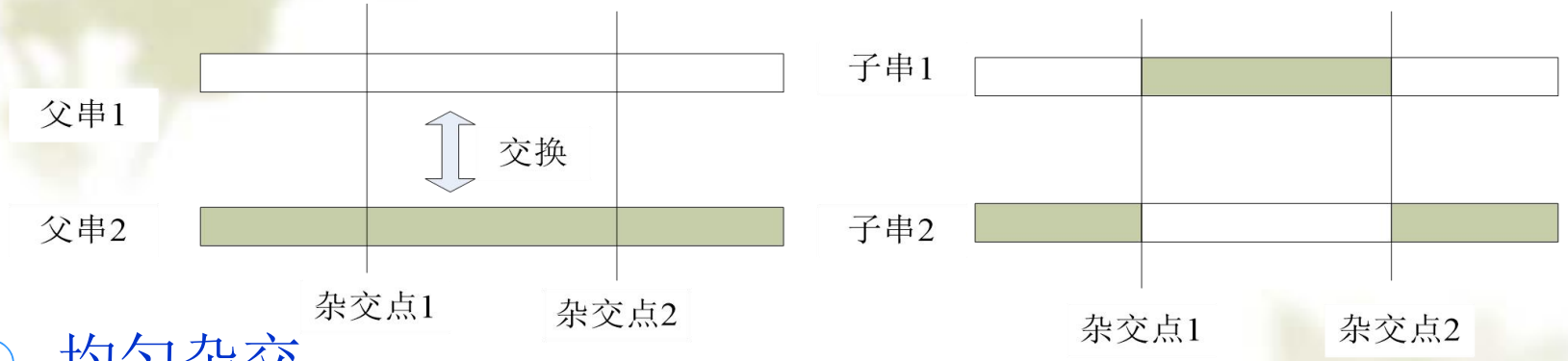
交叉后：

01000|0000011111000101　（孩子1）

11100|0111000000010000　（孩子2）

② 双点杂交与多点杂交

双点杂交是指在个体编码串中设置了二个杂交点，然后再进行部分基因交换。



③ 均匀杂交

- 均匀杂交是指两个配对个体每一个基因座上的基因都以相同的杂交概率进行交换从而形成两个新的个体。
- 均匀杂交实际上可归属于多点杂交的范围。

④ 算术杂交

- 是由两个个体的线性组合而产生出两个新的个体。
- 为了能够进行线性组合运算，算术杂交的操作对象一般是浮点数所表示的个体。

不论是单点交叉还是双点交叉，基因重组后产生的后代可能出现编码重复的情况，需要对产生的子代进行修订。常见的修订算法有PMX，顺序交叉OX和循环交叉CX等。

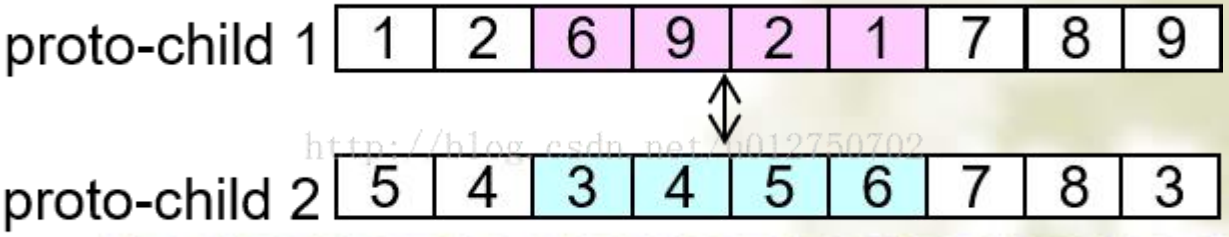
1、Partial-Mapped Crossover (PMX)

过程：

Step1，随机选择一对染色体（父代）中几个基因的起止位置（两染色体被选位置相同）：



Step2，交换这两组基因的位置：



Step3，做冲突检测，根据交换的两组基因建立一个映射关系。

如图所示，以1-6-3这一映射关系为例，可以看到第二步结果中子代1存在两个基因1，这时将其通过映射关系转变为基因3，以此类推至没有冲突为止。最后所有冲突的基因都会经过映射，保证形成的新一对子代基因无冲突：



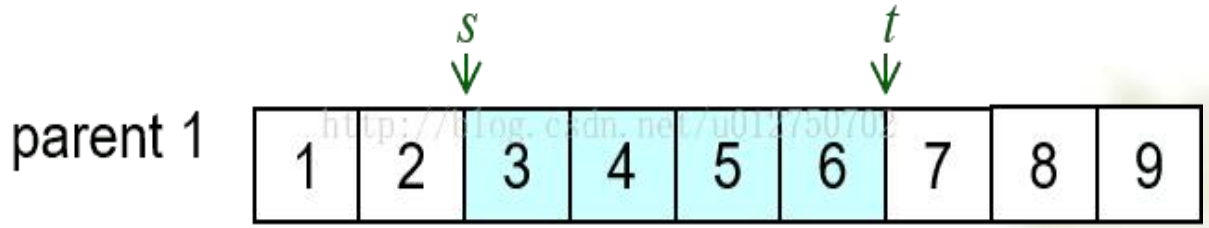
最终结果：



2、Order Crossover (OX)

过程：

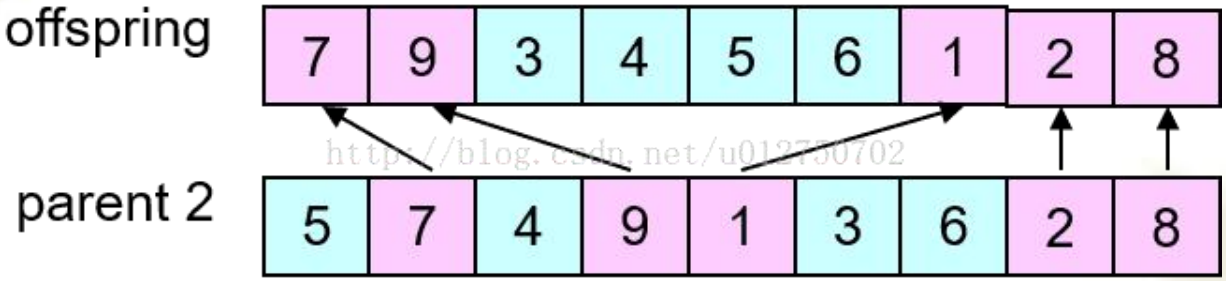
Step1，与PMX相同，随机选择一对染色体(父代)中几个基因的起止位置(两染色体被选位置相同)：



Step2，生成一个子代，并保证子代中被选中的基因的位置与父代相同：



Step 3（可再分两小步），先找出第一步选中的基因在另一个父代中的位置，再将其余基因按顺序放入上一步生成的子代中：



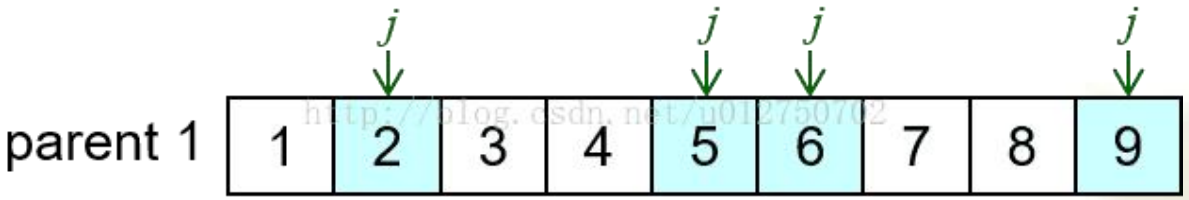
需要注意的是，这种算法同样会生成两个子代，另一个子代生成过程完全相同，只需要将两个父代染色体交换位置，第一步选中的基因型位置相同，本例中的另一个子代为：254913678

与PMX不同的是，不用进行冲突检测工作（实际上也只有PMX需要做冲突检测）。

3、Position-based Crossover (PBX)

过程：

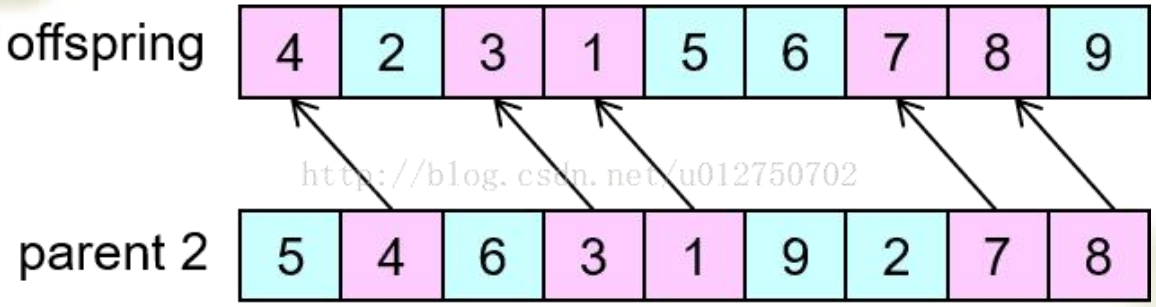
第一步，随机选择一对染色体（父代）中几个基因，位置可不连续，但两染色体被选位置相同：



第二步，与OX的第二步相同，生成一个子代，并保证子代中被选中的基因的位置与父代相同：



第三步，也与OX的第三步相同，先找出第一步选中的基因在另一个父代中的位置，再将其余基因按顺序放入上一步生成的子代中：



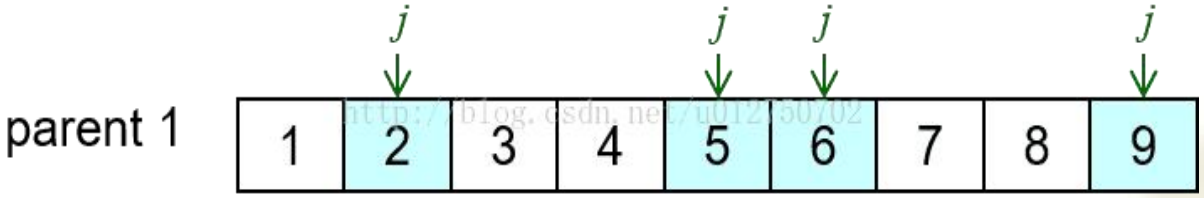
http://blog.csdn.net/u012750702

与上俩个算法不同的是，选择的基因型位置可以不连续，出来这一点外与OX基本一致，同样有两个子代，本例的另一个为：243519678。

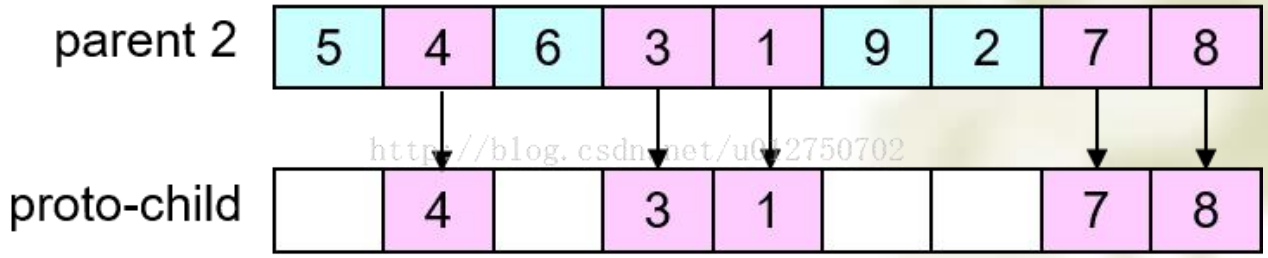
4、Order-Based Crossover (OBX)

过程：

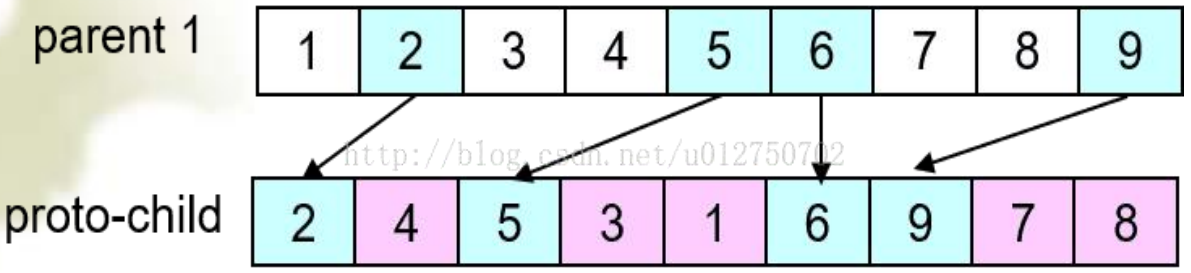
第一步，随机选择一对染色体（父代）中几个基因，位置可不连续，但两染色体被选位置相同：



第二步（分为两小步），先在父代2中找到父代1被选中基因的位置，再用父代2中其余的基因生成子代，并保证位置对应：



第三步，将父代1中被选择的基因按顺序放入子代剩余位置中

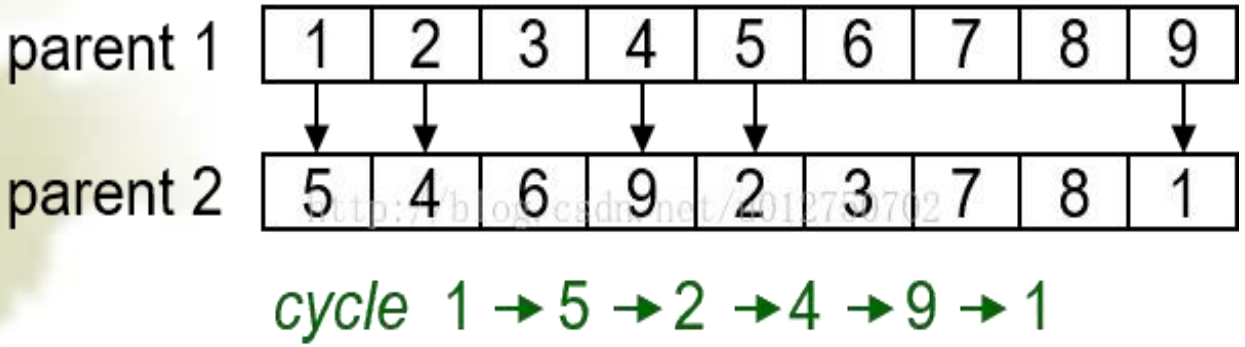


同理，交换父代1、2可得到另一个子代（被选择基因的位置不变），本例结果为：**423156798**。

OBX与**PBX**相比生成子代的“基础”基因来源不同一个来自被选中基因，一个来自剩余的，此外思想基本相似。

5、Cycle Crossover (CX)

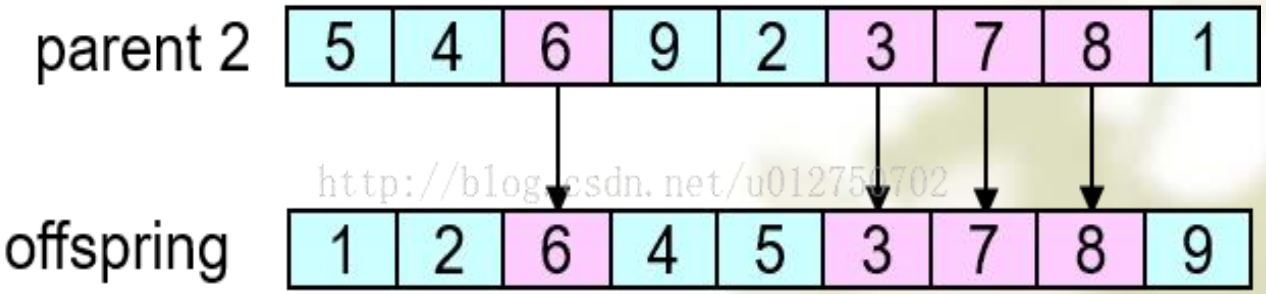
过程：*Step1* 在某个父代上随机选择1个基因，然后找到另一个父代相应位置上的基因编号，再回到第一个父代找到同编号的基因的位置，重复先前工作，直至形成一个环，环中的所有基因的位置即为最后选中的位置：



Step2 用父代1中选中的基因生成子代，并保证位置对应：



Step3 将父代2中剩余基因放入子代中：



与上述算法不同的是，仅需要在一个染色体上随机选择一个位置；本例另一个子代结果为：543926781。

6、Subtour Exchange Crossover

过程：**Step1** 在某个父代上选择1组基因，在另一父代上找到这些基因的位置：

parent 1	1	2	3	4	5	6	7	8	9
parent 2	3	4	9	7	8	5	2	1	6

Step2，保持未选中基因不变，按选中基因的出现顺序，交换两父代染色体中基因的位置，一次生成两个子代：

offspring 1	1	2	3	4	7	5	6	8	9
offspring 2	3	4	9	5	8	6	2	1	7

与上述算法不同的是，只在一个染色体上选择基因的位置。

五、遗传算子的设计

3.变异算子

- 变异运算，是指改变个体编码串中的某些基因值，从而形成新的个体。
- 变异运算是产生新个体的辅助方法，决定遗传算法的局部搜索能力，保持种群多样性。
- 交叉运算和变异运算的相互配合，共同完成对搜索空间的全局搜索和局部搜索。
- 遗传算法中使用变异算子主要有以下两个目的：
  - 改善遗传算法的局部搜索能力。
  - 维持群体的多样性，防止出现早熟现象。

基本遗传算法（SGA）中变异算子采用基本位变异算子。

① 基本位变异

- 对个体的每一个基因座，依变异概率p_m指定其为变异点。
- 对每一个指定的变异点，对其基因值做取反运算或用其他等位基因值来代替，从而产生出一个新的个体。

示例

变异前：

00000111000<u>0</u>0000010000

变异点

变异后：

00000111000<u>1</u>0000010000

1. 整数编码的变异方法

（1）**位点变异**：群体中的个体码串，随机挑选一个或多个基因座，并对这些基因座的基因值以变异概率作变动。

（2）**逆转变异**：在个体码串中随机选择两点（逆转点），然后将两点之间的基因值以逆向排序插入到原位置中。

（3）**插入变异**：在个体码串中随机选择一个码，然后将此码插入随机选择的插入点中间。

（4）**互换变异**：随机选取染色体的两个基因进行简单互换。p_m

（5）**移动变异**：随机选取一个基因，向左或者向右移动一个随机位数。

（6）**自适应变异**：类似于位点变异，但变异概率　随群体中个体的多样性程度而自适应调整。

2. 实数编码的变异方法（略）